

Babet Manual

Contents

Babet — User Manual	5
Getting started	5
Modules	5
Cookbook	5
Building a PDF	5
Conventions used in the docs	6
Development methodology	6
Getting started	6
Download	6
Build from source	6
Three ways to run a Lua script	6
1. Folder mode	6
2. Embedded mode (single-file executable)	6
3. Embedded via PATH (folder + auto-detect)	7
Command-line invocation	7
First script	7
Where to look next	8
Security model	8
What the hardening <i>does</i> protect against	8
What it does <i>not</i> protect against	8
Recommended pattern : least privilege	8
argparse — command-line argument parser (Lua module)	9
Why	9
API	9
opts table fields	9
parse(src?) return contract	10
Source of arguments	10
Accepted argument forms	10
Quick example	10
Error contract	11
Design decisions	11
Not in v1	11
babet.exec — run an external program	12
Why	12
API	12
Quick examples	13
Error contract	13
Design decisions	13
Not in v1	14
babet fs — file system	14

Why	14
API — Existence, type, attributes	14
API — Creating, removing, moving	15
mkdir in detail	15
API — Path manipulation	16
API — Listing and finding	16
API — Copy	16
API — Hashes and checksums	16
API — In-memory checksums	17
Quick examples	17
Error contract	18
Design decisions	18
Not in v1	18
babet.http — HTTP client	18
Why	18
API	18
opts table	19
response table	19
Quick examples	19
Error contract	20
TLS / trust store	20
Design decisions	20
Not in v1	20
babet.inotify — filesystem event watching	21
Why	21
API	21
events argument	21
Event returned by read	21
Quick example	22
Error contract	22
Design decisions	22
Not in v1	23
babet.json — JSON encode/decode	23
Why	23
API	23
Quick example	23
null, empty_array, as_array	24
Error contract	24
Design decisions	24
Not in v1	24
logging — leveled logger (Lua module)	24
Why	24
API	24
Quick example	25
Error contract	25
Design decisions	25
Not in v1	25
babet.signal — POSIX signals	26
Why	26
API	26
Quick example	26
How callbacks run	27

Error contract	27
Design decisions	27
Not in v1	27
babet.socket — TCP client and server	28
Why	28
API	28
Constructors	28
Methods (client socket)	28
Methods (server socket)	28
Quick examples	29
TCP echo client	29
Simple line-protocol server with graceful shutdown	29
Error contract	29
Design decisions	30
Not in v1	30
babet.sqlite — embedded SQL database	30
Why	30
API	30
Opening and closing	30
Execution and queries	31
Type mapping	31
Quick examples	31
Error contract	32
Design decisions	32
Not in v1	32
babet.strings — string manipulation	33
Why	33
API	33
Quick example	33
Error contract	33
Design decisions	33
Not in v1	34
babet.sys — system utilities	34
Why	34
API	34
Runtime constants	34
Quick example	35
Error contract	35
Design decisions	35
Not in v1	36
babet.tables — table manipulation helpers	36
Why	36
API	36
Quick example	36
Error contract	37
Design decisions	37
Not in v1	37
babet.time — monotonic and realtime clocks	37
Why	37
API	38
API — date and duration utilities (v1.8.0+)	38

Quick example	38
Error contract	39
Design decisions	39
Not in v1	39
babel.socket TLS — connect_tls and starttls	39
Why	40
API	40
Quick examples	40
Connect to IRC over TLS	40
STARTTLS upgrade (SMTP submission)	41
Self-signed server (dev / private CA)	41
Skip verification (TESTING ONLY)	41
Error contract	41
Trust store	41
Design decisions	42
Not in v1	42
babel.toml — TOML parser	42
Why	42
API	42
Quick example	42
Error contract	43
Design decisions	43
Not in v1	43
babel.user	43
Why	43
API	43
Returned table on success	44
Quick example	44
Error contract	44
Design decisions	44
Not in v1	44
babel.workers — parallel jobs	45
Why	45
API	45
code argument	45
args argument	45
Quick examples	46
Parallel HTTP fetches	46
CPU-bound work with cancellation	46
Worker pool pattern	47
Error contract	47
Sharing data	47
Design decisions	48
Not in v1	48
Cookbook	48
Watch a directory and email new files	48
Ensure a service user exists, then drop to it	49
Fetch many URLs in parallel	49
Graceful shutdown of a long-running script	49
Read a TOML config, validate, persist to SQLite	50
Cache with a human-readable TTL, expiry as ISO timestamps	50
Pretty-print a Lua value	51

Babet — User Manual

Babet, n.m. — regional word from south-eastern France (Lyonnais, Forez, Dauphiné, Savoie, and adjacent Swiss Romandy) for a pine cone. Small, light, packed with seeds, and good for starting a fire — like this binary.

Babet is a standalone Lua 5.5 binary for Linux scripting and automation. This is the reference manual, organised one file per module so each section stays focused and easy to maintain.

Getting started

- [getting-started.md](#) — install, first script, the three launch modes (folder / embedded / via PATH).
- [security.md](#) — what the hardening does and doesn't protect against, recommended least-privilege patterns.

Modules

Each module lives in its own file under `modules/`:

Module	Purpose
argparse	Command-line argument parser.
exec	Run external programs and capture output.
fs	File system : list, copy, hash, attrs.
http	HTTP client (built on <code>cpp-httplib</code>).
inotify	Filesystem event watching (<code>inotify(7)</code>).
json	JSON encode/decode (<code>nlohmann/json</code>).
logging	Leveled logger.
signal	POSIX signals : <code>SIGTERM</code> , <code>SIGINT</code> , ...
socket	TCP client/server with timeouts.
sqlite	Embedded SQL database.
strings	String manipulation (<code>split</code> , etc.).
sys	<code>which</code> , <code>hostname</code> , <code>env</code> , <code>pid</code> , ...
tables	Table manipulation (<code>mergeTables</code> , <code>deepCopyTable</code>).
time	Monotonic and realtime clocks.
tls	TLS sockets : <code>connect_tls</code> , <code>starttls</code> .
toml	TOML parser (<code>tomlplusplus</code>).
user	System user lookups via NSS.
workers	Parallel jobs (real OS threads).

The [user](#) module is the canonical reference for the documentation pattern : *Why, API, Quick example, Error contract, Design decisions, Not in v1*. Other modules vary in how strictly they follow that template ; the goal over time is to align them.

Cookbook

[cookbook.md](#) — concrete recipes : directory watch + email, parallel HTTP fetches, graceful shutdown, TOML+SQLite, etc.

Building a PDF

The manual can be exported to a single PDF :

```
cd docs
./build_doc.sh          # uses pandoc + LaTeX
```

See `docs/manual_order_en.txt` for the file order ; that's the only place to edit when adding or reordering chapters.

Conventions used in the docs

- **API tables** : every module starts with a small table listing functions, signatures, and what they return.
- **Error contract** : a dedicated section explains which errors are raised (`luaL_error`) versus returned as (`nil`, `err`). This is consistent across all modules.
- **Design decisions** : when a choice is non-obvious (“why not recursive ?”, “why mandatory ?”), there’s a short rationale.
- **Not in v1** : a short list of what could be added additively later without breaking SemVer.

Development methodology

Babet is developed by a single author with substantial AI assistance (primarily Claude / Anthropic, with cross-audits from ChatGPT, Gemini, and Mistral). Every suggestion from these tools is treated as a hypothesis to verify — AI cross-audits regularly identify “bugs” that don’t actually exist, or propose API changes for APIs that haven’t changed. Every change is validated against the 890-test harness before release.

Getting started

Download

Prebuilt binaries for x86_64, aarch64 (RPi4), and armv6l (RPi0) are available on the [releases page](#).

Build from source

Babet vendors all its dependencies (Lua, OpenSSL, SQLite, miniz, nlohmann/json, cpp-httplib, tomlplusplus), so the only prerequisites on your system are :

- a C++23 compiler (recent GCC or Clang)
- CMake (3.20)
- wget and unzip

```
git clone https://github.com/Chipsterjulien/babet.git
cd babet
./build_local.sh           # downloads deps and compiles
                           # (~5 min on first run, faster afterwards)
./run_tests.sh             # offline harness – should print 890 PASS / 0 FAIL
```

The build script downloads each dependency from its upstream source, verifies the SHA256, then compiles statically. If the upstream is temporarily unreachable ([lua.org](#) notably has had outages), the script falls back to the Internet Archive’s Wayback Machine — the SHA256 check still applies.

The resulting binary is at `test/babet`.

Three ways to run a Lua script

Babet supports three launch modes :

1. Folder mode

Point the binary at a directory containing `main.lua` :

```
echo 'print("hello from Babet")' > main.lua
./test/babet .
```

`require("X")` in `main.lua` looks for `X.lua` in the same directory.

2. Embedded mode (single-file executable)

Package a script and its modules into a self-contained binary :

```
./test/babet --create-exe . myapp
./myapp      # runs main.lua from the embedded ZIP
```

Internally, Babet appends a ZIP to its own binary, and reads `main.lua` (plus any required module) from that ZIP at runtime. The resulting `myapp` is fully self-contained — no Babet install needed on the target machine, just glibc.

3. Embedded via PATH (folder + auto-detect)

If Babet is on `$PATH` and you `chmod +x main.lua` after adding a shebang line :

```
#!/usr/bin/env babet
print("hello")

chmod +x main.lua
./main.lua
```

Babet uses the script's own directory as the folder, so `require("helpers")` finds `helpers.lua` next to `main.lua`.

Command-line invocation

In addition to the three launch modes above, Babet accepts a few standard flags :

Flag	Effect
<code>-h, --help</code>	Print the help text and exit 0.
<code>-V, --version</code>	Print <code>babet <version></code> and exit 0.
<code>-c <dir> <out>, --create-exe <dir> <out></code>	Create a self-contained executable named <code><out></code> by embedding <code><dir></code> (which must contain <code>main.lua</code>).

Any other argument starting with `-` is treated as an **unknown option** : Babet prints `Unknown option: ...` + a hint to use `--help`, and exits 1 instead of trying to interpret it as a directory name. Folders whose name legitimately starts with `-` can still be passed via `./-dirname` (POSIX convention).

```
babet --version    # babet 2.0.2
babet --help      # full usage
babet --bogus     # Unknown option: --bogus
                  # Try 'babet --help' for more information.
```

The same version is also exposed to scripts as `babet.VERSION` (plus `babet.VERSION_MAJOR` / `VERSION_MINOR` / `VERSION_PATCH` as integers) — see [sys](#).

First script

Once the binary is built, try this :

```
-- main.lua
print("Babet says hi")
print("PID:", babet.pid())
print("Host:", babet.hostname())

local r, err = babet.http.get("https://example.com/")
if r then
    print("Status:", r.status)
else
    print("HTTP failed:", err)
end
```

Run it with `./test/babet .` (or whichever mode you prefer).

Where to look next

- Each module has its own page under `modules/`.
- The `user` module is a small, complete example of the documentation pattern used throughout — read it as the canonical reference.
- For date and duration utilities (ISO 8601 timestamps, human durations like "5m" or "2h30m"), see `time` — the `babet.time.*` sub-table covers parsing and formatting.
- See `security.md` before exposing Babet scripts to anything that might receive untrusted input.

Security model

Babet runs trusted Lua scripts. It is not a sandbox.

Scripts execute with the full Lua standard library (`luaL_openlibs`), and Babet additionally exposes filesystem and process primitives such as `exec`, `remove`, `rmdirAll` and `chdir`. A script therefore has the same privileges as the process running it : it can read, modify or delete files and run arbitrary commands. This is by design — like `make`, a shell script, or any build tool, Babet is meant to run code you control.

Only run `main.lua` and `required` modules that you trust. Do **not** use Babet to execute Lua from untrusted sources ; it provides no isolation against hostile code, and none is intended. If you need to run untrusted scripts, use a purpose-built Lua sandbox instead.

What the hardening *does* protect against

The hardening work in Babet protects *legitimate* use against accidents and supply-chain tampering :

- **Path / symlink handling** uses `lexically_relative()` and component-wise `is_within()` guards (never string prefix checks), so a `cp src dst/` cannot escape into a sibling tree via a symlinked path component.
- **Process-group cleanup** in `babet.exec` ensures that forked children don't outlive the parent or leak zombie processes when the script is interrupted.
- **Output limits** on `babet.exec` bound the amount of stdout and stderr captured into memory (default 16 MiB each, configurable), so a runaway subprocess can't OOM the Babet process.
- **Dependency checksums** : every vendored dependency is SHA256-pinned in `build_local.sh`. An empty hash refuses to build. A mismatch deletes the downloaded file and exits with an error. This protects against a compromised upstream or Wayback Machine archive (which is the fallback source).
- **TLS verification** is on by default (`verify=true`). Disabling it requires an explicit `verify=false` per call — no silent fallback. Custom CA stores can be passed via `ca_cert` or `ca_path`, or via `SSL_CERT_FILE` / `SSL_CERT_DIR` environment variables.

What it does *not* protect against

- A malicious script. Babet has no sandbox. If you `exec` user input, you have a shell injection. If you `loadstring` user input, you have arbitrary code execution.
- A determined attacker who has obtained code execution on the machine running Babet. The hardening makes accidental mistakes loud, not adversarial attacks impossible.
- Long-tail OS-level issues (kernel exploits, container escapes, privilege escalation). Babet is an ordinary user-land binary.

Recommended pattern : least privilege

When running Babet as a service, treat it like any other unprivileged process :

As root, create a dedicated service user with no shell.

```
useradd --system \
  --home-dir /var/lib/myapp \
  --create-home \
  --shell /usr/sbin/nologin \
  --comment "myapp Babet service" \
  myapp
```

```
# Use babet.user.exists("myapp") in your install script to
# verify the user is provisioned before launching the daemon.
```

Combine with systemd unit file options like `User=myapp`, `PrivateTmp=yes`, `ProtectSystem=strict`, `NoNewPrivileges=yes`, and `CapabilityBoundingSet=` (empty unless you genuinely need a capability). The kernel does much more than Babet can to contain a misbehaving script ; let it.

argparse — command-line argument parser (Lua module)

A declarative argument parser for command-line scripts. Bundled as a pure-Lua module, loaded via `require("argparse")`. Returns results as data (not via prints or `os.exit`) — the script decides what to do.

Why

A Babet script that takes command-line arguments shouldn't have to reinvent argument parsing (positional vs flag, defaults, help text, type coercion, choices) every time. `argparse` makes the common CLI patterns one-liners — and stays out of the script's way by returning everything as data : help is a result, not a side effect ; errors are a return value, not an exception.

API

```
local argparse = require("argparse")
local p = argparse(prog, description?)
```

Builder method	Returns	Use
<code>p:flag(spec, opts?)</code>	<code>p</code> (chainable)	Boolean flag (no value).
<code>p:option(spec, opts?)</code>	<code>p</code> (chainable)	Flag that takes a value.
<code>p:argument(name, opts?)</code>	<code>p</code> (chainable)	Positional argument.
<code>p:parse(src?)</code>	<code>(res, err)</code>	Parse arguments (see below).
<code>p:get_usage()</code>	string	Render the help text manually.

spec for flag and option is a **single string** containing one or more names separated by spaces : `"-v"`, `"--verbose"`, or `"-v --verbose"`. Names must start with `-`.

name for argument is a plain identifier (no leading `-`).

opts table fields

Field	Type	Effect
help	string	Help text shown by <code>get_usage()</code> .
default	any	Value returned when the flag/option/argument is absent.
dest	string	Force the destination key in <code>res</code> . Default: first long name without dashes, else first short name without dash.
required	boolean	Mark as required. For positionals, defaults to <code>true</code> unless <code>default</code> is set.
choices	array of strings	Restrict the raw value to a finite set.
convert	function string -> any	Coerce the raw string. Returning <code>nil</code> is treated as a failure (the <code>convert</code> function may return <code>(nil, "reason")</code>).

`parse(src?)` return contract

- **Success** : (res, nil) — res is a table indexed by **dest**, containing the parsed value (or default) for each declared flag / option / argument.
- **Help requested** : ({ help = true, usage = "<text>" }, nil) when the user passed -h or --help. Help is a success, not an error. The script decides what to do (typically print res.usage and return).
- **Failure** : (nil, err) — human-readable string for any user input problem (unknown option, missing value, invalid choice, conversion error, missing required argument, extra positional).

`parse()` **never raises** on user input — every user error goes through (nil, err). Only **builder misuse** (empty spec, name without -, duplicate name, etc.) raises via `error()`, because that's a programmer bug.

Source of arguments

- `p:parse()` (no argument) reads the global **arg** table, indices 1..n (stopping at the first hole). Indices ≤ 0 (binary path, folder) are ignored — these are Babet's internal slots, not user arguments.
- `p:parse({ "a", "b" })` parses an explicit array. Useful for testing or for parsing arguments that don't come from the shell.

Accepted argument forms

- `--long val`
- `--long=val`
- `-s val`
- `-s=val`
- `--` ends option parsing — every subsequent token is treated as a positional, even if it starts with -.

Short options collapsed together (`-abc`) and the `-fvalue` form (short option immediately followed by its value, no separator) are **not supported** in v1.

Quick example

```
local argparse = require("argparse")

local p = argparse("myapp", "Process some files.")
:flag("-v --verbose", { help = "Verbose output" })
:option("-o --output",
        { help = "Output file", default = "out.txt" })
:option("-n --count",
        { help = "How many", convert = tonumber, default = 1 })
:argument("input", { help = "Input file" })

local args, err = p:parse()

-- Help is a success, not an error.
if args and args.help then
    print(args.usage)
    return
end

if err then
    io.stderr:write("error: ", err, "\n")
    io.stderr:write(p:get_usage(), "\n")
    os.exit(2)
end

print(args.input, args.output, args.count, args.verbose)
```

Running it :

```
$ ./myapp.lua data.csv --count 5 -v
data.csv    out.txt    5      true
```

```
$ ./myapp.lua --help
Usage: myapp [options] <input>
```

Process some files.

Arguments:
input Input file

Options:
-h, --help show this help
-v, --verbose Verbose output
-o, --output <value> Output file
-n, --count <value> How many

Error contract

- **Builder misuse** (`p:flag("")`, `p:option("foo")` without `-`, duplicate name, `p:argument("-bad")`) → raises via `error()`. These are programmer bugs.
- **`p:parse(...)` never raises on user input.** All user input problems are reported as `(nil, "message")` :
 - "unknown option '--xxx'"
 - "option '--xxx' requires a value"
 - "flag '-v' does not take a value"
 - "missing required option '--xxx'"
 - "missing required argument 'name'"
 - "argument 'name': invalid choice 'xxx'"
 - "option '--xxx': <converter message>"
 - "unexpected argument 'xxx'"
- A convert function that itself raises is **caught** by `parse()` and turned into a `(nil, "<...>: conversion error")`. The script never sees an exception coming out of `parse()`.

Design decisions

- **Help is a success, not a side effect.** Returning `{ help = true, usage = "..."` } instead of printing and exiting lets the script decide : print to stdout, log to a file, send to a GUI, prepend a banner, whatever. The library doesn't decide for you, consistent with the rest of Babet where no module unilaterally exits the process.
- **(res, err) everywhere on user input, error() only on builder bugs.** Same convention as the rest of Babet (user, json, sqlite, etc.).
- **Spec is a single string** ("`-v --verbose`"), not a chain of calls. Discoverable, terse, and avoids ambiguity when multiple names are declared together.
- **choices is checked on the raw string, then convert applied.** If you pair them, express choices as strings — e.g. `{ choices = {"1","2","3"}, convert = tonumber }`. Documented trap, but conscious : doing the reverse would force choices values to be of the converted type, which couples two independent features.
- **Positional with default is implicitly optional.** Saves the `required = false` boilerplate in the common case.
- **`parse()` reads `arg[1..n]`** — never `arg[0]` or negative indices, where Babet puts its own slots. Just what the user actually typed.

Not in v1

Additive — could be added later without breaking SemVer :

- Sub-commands (`git commit` / `git push` style).
- Combined short options (`-abc` for `-a -b -c`).

- Short options with attached values (`-fvalue`).
- Variadic positionals (`nargs = "+"` or `"*"`).
- Argument groups (visual grouping in help).

babet.exec — run an external program

Spawn a subprocess, capture its stdout / stderr, and get its exit code back as structured data. Where `os.execute` gives you just an integer, **babet.exec** gives you a result table with everything you actually need.

Why

Almost every non-trivial script eventually shells out — to `git`, `useradd`, `convert`, `ffmpeg`, whatever. `os.execute` is too crude (no captured output, shell quoting hazards), and `io.popen` doesn't let you both write to stdin and read stdout cleanly, doesn't expose the exit code without parsing, and has no timeout.

babet.exec is the structured alternative : `argv` passed as a list (no shell, no quoting bugs), output captured into memory with a bounded cap, optional timeout, separate stdout/stderr.

API

```
result, err = babet.exec(cmd, args?, opts?)
```

Argument Type		Notes
<code>cmd</code>	string	Program to run. PATH is searched.
<code>args</code>	table (optional)	Array of string arguments — each one becomes a separate argv slot, no shell parsing.
<code>opts</code>	table (optional)	See below.

`opts` fields :

Field	Type	Default	Notes
<code>cwd</code>	string	<code>inherit</code>	Working directory for the child.
<code>env</code>	table { <code>KEY</code> = <code>value</code> , ...}	<code>inherit</code>	Merged with the parent environment, never replacing it. Override : pass the key. Empty : pass "" (the child sees it empty). Omitting a key = inherit ; there is no unset in v1.
<code>stdin</code>	string	""	Data sent to the child's stdin.
<code>timeout</code>	number (s)	<code>none</code>	On expiry : SIGTERM to the whole process group (clean shutdown possible), 2 s grace, then SIGKILL .
<code>max_output</code>	integer (bytes)	10 MiB	Hard cap per stream on captured output (upper bound : 2 GiB). Excess is silently dropped and signalled via <code>*_truncated</code> .

Result table on successful launch :

```
{
  stdout = "...",           -- captured stdout, up to max_output bytes
  stderr = "...",           -- captured stderr, up to max_output bytes
  code = 0,                 -- exit code (0..255)
  timed_out = false,        -- true if killed by the timeout
  stdout_truncated = false, -- true if more than max_output was produced
  stderr_truncated = false,
}
```

Quick examples

```
-- Basic capture
local r = assert(babet.exec("git", { "rev-parse", "HEAD" }))
if r.code == 0 then
    print("HEAD:", r.stdout:gsub("\n$", ""))
end

-- Non-zero exit is NOT an error
local r = assert(babet.exec("grep", { "needle", "/etc/passwd" }))
if r.code == 0 then
    print("found:", r.stdout)
elseif r.code == 1 then
    print("not found")
else
    print("grep failed:", r.stderr)
end

-- Pipe data to stdin
local r = assert(babet.exec("sha256sum", { "-" }, {
    stdin = "hello world\n",
}))
print(r.stdout)
-- a948904f2f0f479b8f8197694b30184b0d2ed1c1cd2a1ec0fb85d299a192a447 -

-- Augment env (not replace)
local r = assert(babet.exec("env", nil, {
    env = { MY_FLAG = "yes" },
}))
-- Child sees MY_FLAG=yes plus everything the parent already had.

-- Timeout + truncation
local r = assert(babet.exec("yes", nil, {
    timeout = 0.5,
    max_output = 1024,
}))
print(r.timed_out, r.stdout_truncated) -- true true

-- Different cwd
local r = assert(babet.exec("ls", nil, { cwd = "/var/log" }))
```

Error contract

- **(nil, err)** only if the launch itself fails : program not found, cwd doesn't exist, fork failed, OOM, etc.
- **(result, nil)** for every other case, including :
 - The program ran and exited non-zero → `result.code > 0`.
 - The program was killed by the timeout → `result.timed_out` is true, `result.code` reflects the signal exit.
 - The program produced more output than `max_output` → the relevant `*_truncated` flag is true, captured data stops at the cap.
- **opts validation** : invalid types (`opts.cwd` not a string, `opts.env` not a table, `opts.env` keys with = or NUL, `opts.timeout` 0 or NaN/Inf, `opts.max_output` non-integer or > 2 GiB) → raise via `luaL_error`. These are caller bugs.

Design decisions

- **Exit code error.** The most common bug with `os.execute` / `exec`-wrappers is treating non-zero exit as an exception. Many programs use exit codes for meaningful state (`grep` returns 1 when no match, `diff` returns

- 1 when files differ, etc.). The caller decides what counts as failure.
- **No shell.** `args` is an array, each element is one `argv` slot, no quoting needed and no `$(rm -rf $HOME)` surprises. If you really want a shell, pass `"sh" + { "-c", "your command" }`.
- **env merges with parent.** Replacing the full environment is rarely what you want and usually breaks programs that expect `PATH` or `LANG`. To override one variable, pass it. To unset one, pass it as the empty string and the child will see it empty (POSIX doesn't have "unset" semantics at `exec` time without extra effort).
- **SIGTERM first, SIGKILL second.** On timeout, the whole process group (the child creates its own via `setpgid`) gets `SIGTERM` — well-behaved programs can flush and exit — then `SIGKILL` after a 2 s grace period. `kill(-pid)` also covers descendants (a forked `sh -c`, for instance).
- **max_output is per stream, not total.** 10 MiB stdout + 10 MiB stderr is the cap. Large enough to capture most build logs, small enough that a runaway subprocess can't OOM the Babet process.
- **Hard 2 GiB cap on max_output.** Pretty much anything that needs more than 2 GiB of captured stdout should redirect to a file via `opts.stdin` + a shell wrapper, or via `exec("sh", { "-c", "your-cmd > /tmp/log" })`.
- **stdin is a string, not a callback.** Streaming gigabytes into a child is rare and out of scope ; the common case is "feed this small input and read the answer".

Not in v1

- **Interruption by `babet.signal` while running.** `exec` is not signal-aware : a handled signal received while a child runs does not return (`nil`, `"interrupted"`) — the callback is dispatched after `exec` returns. Use `opts.timeout` to bound long runs.
- Streaming stdout / stderr (line-by-line callback during execution). Currently the whole capture is returned at once.
- Stdin streaming (callback that produces input chunks).
- Direct fd redirection (`stdout = "/tmp/log"`). Easy to add later if needed ; the shell-wrapper workaround above covers most cases.
- Process group management beyond the child. The current implementation does correctly tear down the child's process group on timeout / interrupt — see [security](#).

babet fs — file system

A flat set of file system operations exposed directly on `babet` (no sub-namespace, predates the convention). Covers the everyday needs that Lua's `os.*` and `io.*` don't : recursive listing, copy with attributes, hashing, path manipulation, attributes, links.

Why

Lua's standard `io` and `os` give you `open`, `rename`, `remove`, and that's about it. Everything beyond that — listing a directory, recursive copy, computing a hash, querying `mtime` — requires external programs or low-level FFI. `babet` collects these in a single set of functions that behave consistently (paths are strings, errors come back as `(nil, "...")`).

All paths are accepted as plain strings. Tilde expansion (`~/x`) is **not** automatic — the caller must resolve `$HOME` itself if needed. Functions never expand globs ; for glob support, use `babet.find` with an explicit pattern.

API — Existence, type, attributes

Function	Returns
<code>babet.fileExists(path)</code>	boolean
<code>babet.isFile(path)</code>	boolean (true only for regular files)
<code>babet.isDir(path)</code>	boolean
<code>babet.fileSize(path)</code>	integer bytes <code>(nil, err)</code>
<code>babet.getAttributes(path)</code>	table with <code>mtime</code> , <code>mode</code> , <code>size</code> , <code>uid</code> , <code>gid</code> , <code>inode</code> , etc. <code>(nil, err)</code>
<code>babet.setAttributes(path, uid, gid, mode?)</code>	<code>(true, nil)</code> <code>(nil, err)</code> — set owner/group, optionally mode

Function	Returns
<code>babet.symlinkAttr(path, uid, gid)</code>	<code>(true, nil) (nil, err)</code> — change owner/group of the link itself (<code>lchown</code> , does not follow the symlink) ; counterpart of <code>setAttributes</code>
<code>babet.getMode(path)</code>	octal integer <code> (nil, err)</code>
<code>babet.setMode(path, mode)</code>	<code>(true, nil) (nil, err)</code> — <code>mode</code> is an integer (use <code>0xled</code> for <code>0755</code> , or build it with <code>tonumber("755", 8)</code>)

Canonical naming : camelCase for Babet compounds (`isFile`, `isDir`, `symlinkAttr`), lowercase for POSIX-inherited names (`mkdir`, `chdir`, `touch`...). The old names `isfile`, `isdir` and `symlinkattr` remain **deprecated** aliases of the same functions.

API — Creating, removing, moving

Function	Returns
<code>babet.touch(path)</code>	<code>(true, nil) (nil, err)</code> — create file or update mtime
<code>babet.mkdir(path)</code>	<code>(true, nil) (nil, err)</code> — always recursive (native <code>mkdir -p</code>), see below
<code>babet.rmdir(path)</code>	<code>(true, nil) (nil, err)</code> — empty directories only
<code>babet.rmdirAll(path)</code>	<code>(true, nil) (nil, err)</code> — recursive
<code>babet.remove(path)</code>	<code>(true, nil) (nil, err)</code> — file or symlink
<code>babet.rename(old, new)</code>	<code>(true, nil) (nil, err)</code>
<code>babet.chdir(path)</code>	<code>(true, nil) (nil, err)</code>

`chdir` mutates the working directory of the **WHOLE** process : it is forbidden once a `workers.spawn` has happened `((nil, err), permanent)`. See the note in [sys](#).

Function	Returns
<code>babet.currentDir()</code>	string (absolute)
<code>babet.link(target, linkpath)</code>	<code>(true, nil) (nil, err)</code> — creates a symbolic link (always ; no hard links in v1)

mkdir in detail

`babet.mkdir(path)` guarantees the directory exists when the call returns :

- **Always recursive** : missing parents are created in one go (native `mkdir -p` — there is **no** option to pass, contrary to what an earlier version of this page claimed).
- **Idempotent** : if the directory (or a symlink to a directory) already exists, silent success.
- **Clear error** when a non-directory blocks the path — a file at the target path or in the middle of it : `(nil, err)` containing “Not a directory”.

```
-- creates data/, data/cache/ and data/cache/img/ in one call
assert(babet.mkdir("data/cache/img"))
```

```
-- calling it again costs nothing : already there, success
assert(babet.mkdir("data/cache/img"))
```

```
-- a file blocks the path -> clean error
babet.touch("blocker")
local r, err = babet.mkdir("blocker/sub")
-- r == nil, err contains "Not a directory"
```

Legacy signature : a second boolean parameter still exists (`mkdir(path, ignore_if_exists)`) but it is **discouraged** : on Linux its only observable effect is turning the “a dangling symlink occupies the path” error into silent success... without creating anything. Don’t use it.

API — Path manipulation

These operate on path *strings* — they don't touch the filesystem.

Function	Returns	Example
<code>babet.getBaseName(path)</code>	<code>string</code> <code>(nil, err)</code> — last component	<code>"/etc/hostname" → "hostname"</code>
<code>babet.getPath(path)</code>	<code>string</code> <code>(nil, err)</code> — parent dir (dirname)	<code>"/etc/hostname" → "/etc"</code>
<code>babet.getFilename(path)</code>	<code>string</code> <code>(nil, err)</code> — basename without extension	<code>"report.tar.gz" → "report.tar"</code>
<code>babet.getExtension(path)</code>	<code>string</code> <code>(nil, err)</code> — final extension	<code>"report.tar.gz" → ".gz"</code>

API — Listing and finding

Function	Returns
<code>babet.listFiles(path)</code>	<code>table</code> (regular files in path) <code>(nil, err)</code>
<code>babet.find(path, opts)</code>	iterator function, see below
<code>babet.createFileIterator(path)</code>	iterator function over directory entries

`find` accepts `opts` :

- `pattern = "*.lua"` — shell glob applied to file name only.
- `recursive = true` — descend into sub-directories.
- `type = "f" / "d"` — restrict to files or dirs.
- `max_depth = N` — limit recursion depth.

API — Copy

Function	Returns
<code>babet.copy(src, dst, opts?)</code>	<code>(true, nil)</code> <code>(nil, err)</code>
<code>babet.copyTree(src, dst, opts?)</code>	<code>(true, nil)</code> <code>(nil, err)</code> — recursive
<code>babet.moveTree(src, dst)</code>	<code>(true, nil)</code> <code>(nil, err)</code>

copy options :

- `preserve = true` — keep mtime, mode, owner where possible.
- `overwrite = true` — replace destination if it exists.

API — Hashes and checksums

Hash digests of file contents. Result is **lowercase hex string** unless noted. Note the `sum` suffix — these are the registered names in `babet.*`.

Function	Algorithm	Notes
<code>babet.crc32sum(path)</code>	CRC-32 (IEEE 802.3)	Not cryptographic. For integrity / accidental-error detection only.
<code>babet.md5sum(path)</code>	MD5	Legacy. Don't use for security.
<code>babet.sha1sum(path)</code>	SHA-1	Legacy. Don't use for security.
<code>babet.sha256sum(path)</code>	SHA-256	
<code>babet.sha384sum(path)</code>	SHA-384	
<code>babet.sha512sum(path)</code>	SHA-512	
<code>babet.sha3_256sum(path)</code>	SHA3-256	
<code>babet.sha3_384sum(path)</code>	SHA3-384	

Function	Algorithm	Notes
<code>babet.sha3_512sum(path)</code>	SHA3-512	
<code>babet.blake2b512sum(path)</code>	BLAKE2b-512	

Each returns `string` (hex) or `(nil, err)`. The file is streamed, not loaded entirely into RAM. Non-regular files (directories, sockets, ...) are rejected with `(nil, "<algo>: not a regular file")`.

MD5 and SHA-1 are exposed for compatibility (Git, legacy systems) — don't use them for new cryptographic uses. **CRC-32 is even further from cryptographic**: an attacker can produce collisions trivially. Use it for integrity / change detection, never for authentication.

API — In-memory checksums

Compute a hash of bytes already in memory, without round-tripping through a temporary file. Binary-safe (embedded NULs are allowed).

Function	Returns
<code>babet.crc32(data)</code>	<code>string</code> — 8-char lowercase hex

Cannot fail at runtime (pure computation). Wrong argument types raise via `luaL_error`. As with `crc32sum`, **not cryptographic**.

Quick examples

```
-- Predicates
if babet.isDir("/etc") and babet.fileExists("/etc/hostname") then
    print("size:", babet.fileSize("/etc/hostname"))
end

-- Path manipulation (pure string ops)
local base = babet.getBaseName("/var/log/syslog.1") -- "syslog.1"
local dir  = babet.getPath("/var/log/syslog.1")    -- "/var/log"
local ext  = babet.getExtension("/var/log/syslog.1") -- ".1"

-- Recursive listing
for entry in babet.find("/var/log",
    { pattern = "*.log", type = "f", recursive = true }) do
    print(entry)
end

-- Copy with attributes
babet.copyTree("./src", "/tmp/backup",
    { preserve = true, overwrite = true })

-- Hash a release tarball
local sha, err = babet.sha256sum("/tmp/babet-1.7.0.tar.gz")
if sha then print("SHA256:", sha) end

-- Quick integrity check (NOT cryptographic, but fast)
local crc = babet.crc32sum("/tmp/babet-1.7.0.tar.gz")
print("CRC32:", crc) -- e.g. "a1b2c3d4"

-- In-memory CRC32 of arbitrary bytes (binary-safe, NULs OK)
print(babet.crc32("abc")) -- "352441c2"
```

```

print(babet.crc32(""))          -- "00000000"

-- Set ownership and permissions
local mode_755 = tonumber("755", 8) -- octal -> integer
assert(babet.setMode("/srv/myapp/run.sh", mode_755))
-- root only:
-- assert(babet.setAttributes("/srv/myapp", 1000, 1000))

```

Error contract

- All functions follow the `(nil, err)` convention on runtime failure (no such file, permission denied, etc.).
- Wrong argument **types** raise via `luaL_error`.
- **setAttributes(path, uid, gid)** rejects negative UIDs/GIDs and values above `uid_t/gid_t` max (typically $2^{32} - 1$). Without that upper-bound check, a value above the limit would truncate silently — worst case to UID 0 (root). Same logic as the `user` module.
- Path safety : `copy`, `copyTree`, `moveTree` use `lexically_relative()` and component-wise `is_within()` guards to prevent symlink-based escapes out of the destination tree. See [security](#).

Design decisions

- **Flat namespace** : `fileExists` instead of `fs.exists`, `getAttributes` instead of `fs.attr.get`. Pure historical reason — these functions predate the per-module convention. Stable now ; renaming would break every script.
- **get* / set* for path attributes** (`getMode/setMode`, `getAttributes/setAttributes`) but plain verbs for FS actions (`remove`, `touch`, `mkdir`). The “get/set” pairs exist because both directions are needed ; the others are single-direction operations.
- **Path manipulation is separate from FS access**. `getBasename`, `getPath`, etc. operate on strings only — they don’t stat the path. Use `fileExists` if you need to know whether the path actually exists.
- **Hashes have a sum suffix** to match the standard Unix tools (`md5sum`, `sha256sum`, etc.). Discoverable for anyone familiar with the command line.
- **copy is single-file, copyTree is recursive**. Lua doesn’t have function overloading by type, so splitting them avoids the ambiguity “did `copy('dir', 'dir')` mean recursive or not ?”.

Not in v1

- Glob matching as a standalone function (`babet.glob`). Use `find` with `pattern = "*.lua"` for the common case.
- Async I/O. Use [workers](#) for parallel file processing instead.
- BLAKE2s (only BLAKE2b-512 is exposed via OpenSSL EVP).

babet.http — HTTP client

A simple HTTP/HTTPS client built on [cpp-httplib](#), reusing the vendored OpenSSL. Synchronous, blocking, with timeouts.

Why

Almost every non-trivial script needs to talk to some HTTP API. Without a built-in client, you reach for `curl` via `exec`, with all the quoting and process-spawning overhead. `babet.http` makes a request a one-liner, with proper TLS verification by default.

API

Function	Returns
<code>babet.http.request(opts)</code>	<code>response (table) (nil, err)</code> — methods : GET, HEAD, OPTIONS, POST, PUT, PATCH, DELETE
<code>babet.http.get(url, opts?)</code>	shortcut for <code>request{ method="GET", url=url, ... }</code>

Function	Returns
<code>babet.http.post(url, opts?)</code>	shortcut for <code>request{ method="POST", url=url, ... }</code>

No put/delete/patch shortcuts in v1 (an earlier version of this page wrongly advertised them) : use `request{ method = "PUT", ... }`. See “Not in v1”.

opts table

Field	Type	Default
<code>url</code>	string (required for <code>request</code>)	—
<code>method</code>	string — GET, HEAD, OPTIONS, POST, PUT, PATCH, DELETE	"GET"
<code>headers</code>	table of <code>name = value</code>	{}
<code>body</code>	string (forbidden for GET/HEAD/OPTIONS)	""
<code>query</code>	table { <code>key = value</code> } — URL-encoded and appended to the URL (<code>?a=b&c=d</code>), merges cleanly with an existing query	—
<code>timeout</code>	number (seconds)	none — without it, <code>cpp-httpplib</code> 's internal defaults apply ; always pass one
<code>verify</code>	boolean (TLS cert verification)	true
<code>ca_cert</code>	string (path to CA bundle file)	system default
<code>follow_redirects</code>	boolean	false — following is opt-in ; hop limit internal to <code>cpp-httpplib</code>

response table

```
{
  status = 200,
  body = "...",
  headers = {
    -- normalised lowercase keys
    ["content-type"] = "application/json",
    ["content-length"] = "42",
  },
}
```

Duplicate response headers : the last value wins (one key = one string). Known consequence : if a server sends several `Set-Cookie` headers, only the last one is visible. If that need becomes real, `headers` could carry a table of strings for repeated keys — see “Not in v1”.

Quick examples

```
-- Simple GET
local r, err = babet.http.get("https://api.example.com/health")
if r then
  print(r.status, r.body)
end

-- JSON POST with auth header
local r, err = babet.http.post("https://api.example.com/v1/things", {
  headers = {
    ["Content-Type"] = "application/json",
    ["Authorization"] = "Bearer " .. token,
  },
  body = babet.json.encode({ name = "widget", count = 7 }),
  timeout = 10,
```

```

})

-- Self-signed cert (dev / private CA)
local r = babet.http.get("https://internal.svc/", {
    ca_cert = "/etc/myapp/internal-ca.crt",
})

-- Disable verification (TESTING ONLY)
local r = babet.http.get("https://expired.badssl.com/",
    { verify = false })

```

Error contract

- **Wrong argument types** → raises via `luaL_error`.
- **Network errors** (DNS, connect, timeout, TLS) → `(nil, "http: <description>")`.
- **body on GET/HEAD/OPTIONS** → `(nil, "http: body not allowed for <method>")` — reported, never silently dropped.
- **Unknown method** → `(nil, "http: unsupported method '...')"`.
- **HTTP status errors** are **not** errors — a 404 returns a normal response table with `status = 404`. Status semantics are the caller's responsibility.

TLS / trust store

Babet ships its own static OpenSSL, so it doesn't automatically inherit the distro's `ca-certificates` configuration. Two mechanisms ensure `verify=true` works out of the box :

1. The vendored OpenSSL is built with `--openssldir=/etc/ssl`, covering Arch, Debian, Ubuntu, Alpine, Gentoo.
2. `babet.socket.connect_tls` additionally probes the known CA bundle paths (Fedora/RHEL, OpenSUSE, FreeBSD, NetBSD). **babet.http relies on the OpenSSL defaults alone** (mechanism 1) : on an exotic layout where point 1 is not enough, `http` may need `ca_cert` where `connect_tls` works. `tools/verif_ca.lua` diagnoses both paths.

You can override per-call with `ca_cert`, or globally via the `SSL_CERT_FILE` / `SSL_CERT_DIR` environment variables. [security](#) and [tls](#) for details.

Design decisions

- **Synchronous, blocking.** Scripts are usually one-shot request-response affairs ; sync is simpler and sufficient. For many parallel calls, use [workers](#).
- **verify=true by default.** Disabling TLS verification must be explicit (`verify=false`). No silent downgrade.
- **HTTP status is not an error.** The caller decides whether 404 or 500 is failure ; the network call itself succeeded.
- **Headers are normalised to lowercase keys** in the response, to make case-insensitive lookups work directly (`r.headers["content-type"]`).
- **Redirects are opt-in.** An unfollowed redirect is visible (`status = 301/302 + location` header) ; following it is the caller's decision (`follow_redirects = true`). The hop limit is `cpp-httpplib`'s, not ours.

Not in v1

- `put` / `delete` / `patch` shortcuts — covered by `request{ method = ... }` ; trivial sugar to add if the need is confirmed.
- `opts.ca_path` (per-call CA directory) and `opts.max_redirects` — an earlier version of this page wrongly documented them.
- Repeated response headers as a table of strings (multiple `Set-Cookie` case).
- Streaming response body (e.g. for downloading large files). Currently `body` is read fully into memory.
- HTTP/2 / HTTP/3.
- WebSockets (use [socket](#) + TLS + a Lua framing library if needed).
- Server side. `cpp-httpplib` supports it, but exposing a robust HTTP server to Lua is its own design effort.

babet.inotify — filesystem event watching

Wraps Linux `inotify(7)` for instant filesystem event delivery — no polling. Used for hot-reloading configuration, watching drop directories, log tailing, file synchronisation triggers, etc.

Why

Polling a directory with `listDir` every second is wasteful (CPU + inode lookups) and laggy. `inotify` is the kernel's purpose-built mechanism : the kernel wakes your script only when something actually changes, with sub-millisecond latency.

The Lua API exposes a minimal contract over a notoriously tricky syscall. Queue overflow is surfaced explicitly (the only thing worse than dropping events silently is not telling you about it).

API

Function	Returns
<code>babet.inotify.new()</code>	<code>watcher (userdata) (nil, err)</code>
<code>w:add(path, events)</code>	<code>integer wd (watch descriptor) (nil, err)</code>
<code>w:remove(wd)</code>	<code>(true, nil) (nil, err)</code>
<code>w:read(timeout?)</code>	<code>table of events (nil, "timeout") (nil, "interrupted") (nil, err)</code>
<code>w:close()</code>	<code>(true, nil) — idempotent</code>

`read(timeout?)` : omitted = **blocks** until the next event ; `0` = **non-blocking** (returns the already-pending events, otherwise `(nil, "timeout")`) ; `t > 0` = waits at most `t` seconds. Note : this convention diverges from `socket` where `set_timeout(0)` means infinite — here, “give me what's there right now” is the dominant use case.

events argument

A strict 1..N array of event name strings. Acceptable names :

Name	Triggered when
"access"	file accessed
"modify"	file content modified
"attrib"	metadata changed (perms, mtime, ...)
"close_write"	file opened for writing is closed
"close_nowrite"	read-only fd closed
"close"	combined alias : <code>close_write</code> or <code>close_nowrite</code>
"open"	file opened
"moved_from"	file moved out of watched dir
"moved_to"	file moved into watched dir
"move"	combined alias : <code>moved_from</code> or <code>moved_to</code>
"create"	file/dir created in watched dir
"delete"	file/dir deleted from watched dir
"delete_self"	the watched path itself is deleted
"move_self"	the watched path is moved

Event returned by read

```
{  
  wd      = 1,           -- watch descriptor  
  name    = "newfile.txt", -- " if the watched path itself is the target  
  events  = { create = true, close_write = true },  
  is_dir  = false,
```

```

    cookie = 0,                -- non-zero pairs moved_from / moved_to
}

```

Special synthetic event for **queue overflow** :

```
{ wd = -1, events = { overflow = true } }
```

When you see this, the kernel queue ran out of space and some events were lost. Re-scan the watched paths to recover state.

Quick example

```

local w = assert(babet.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))

babet.signal.handle("TERM", function() w:close(); os.exit(0) end)

while true do
    local events, err = w:read()    -- blocks until something happens
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify: ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            rescan()                -- queue overflowed, recover
        else
            handle_new_file("/srv/incoming/" .. ev.name)
        end
    end
end
end

```

Error contract

- **read(t)** where t is NaN/Inf/negative → raises via `luaL_error`.
- **add(path, events)** with a non-list `events` table (sparse, extra keys, non-string elements) → (nil, err).
- **add on a non-existent path** → (nil, "no such file or directory").
- **read** :
 - `events` table on success.
 - (nil, "timeout") if timeout elapsed.
 - (nil, "interrupted") if a handled signal arrived.
 - (nil, err) on other errors.
- **Methods after close** → (nil, "inotify: closed").

Design decisions

- **Non-recursive in v1.** `inotify(7)` itself isn't recursive ; watching a tree means walking it and calling `add` on each sub-directory, then handling `create` events to extend the watch. That's a real design effort with subtleties (races between walking and event delivery, overflow handling) and worth a dedicated module if/when needed. Buildable in pure Lua on top.
- **Events list mandatory, no implicit "all".** Watching everything floods the queue and forces every event to traverse Lua. Forcing the caller to pick events makes the cost visible.
- **Overflow surfaced explicitly**, never swallowed. The kernel has a finite queue (`/proc/sys/fs/inotify/max_queued_events`, default 16384). When it overflows, events are lost. The only appropriate action is to re-scan ; silently dropping the signal would defeat the entire reliability story.
- **read() is interruptible by signal.** A `read()` blocking forever ignoring `SIGTERM` is the classic graceful-shutdown bug. See [signal](#).
- **IN_CLOEXEC** on the fd : not inherited by exec'd subprocesses, consistent with sockets.

Not in v1

- Recursive watching. Add at the script level (walk + add) or wait for a `recursive = true` option.
- Multiple watchers per process. Doable today by creating multiple `inotify.new()` instances ; not a built-in registry.
- `IN_MASK_ADD` semantics (adding events to an existing watch). Currently `add(path, events)` replaces the mask. Rare need.

babet.json — JSON encode/decode

Backed by [nlohmann/json](#), the most widely-used C++ JSON library. Handles the usual round-trip plus the explicit “empty array” sentinel that Lua tables can’t disambiguate from “empty object”.

Why

The Lua/JSON impedance mismatch always trips someone up : Lua has no distinction between an empty list and an empty table. Most ad-hoc Lua JSON libraries pick one and get it wrong half the time. `babet.json` is explicit : `{}` decodes to `{}` and re-encodes to `{}` (object), but if you want `[]` you use the `babet.json.empty_array` sentinel.

API

Function	Returns
<code>babet.json.encode(value, opts?)</code>	<code>string</code> (JSON text) <code>(nil, err)</code>
<code>babet.json.decode(text)</code>	<code>value</code> <code>(nil, err)</code>
<code>babet.json.null</code>	sentinel — represents JSON null (see below)
<code>babet.json.empty_array</code>	sentinel that encodes as <code>[]</code>
<code>babet.json.as_array(t)</code>	<code>t</code> , tagged to encode as a JSON array even when ambiguous/empty

Encoding options (`opts` table) :

- `indent = N` (integer ≥ 0) — enables pretty-printing with `N` spaces of indentation (`0` = newlines without spaces). Omitted or negative : **compact** output. (An earlier version of this page documented a **pretty** option that never existed : `indent` alone drives everything.)

Quick example

```
local J = babet.json

-- Decode
local data, err = J.decode([[
{ "name": "alice", "tags": ["admin", "ops"], "active": true }
]])
print(data.name, data.tags[1])

-- Encode (compact)
local out = J.encode({ a = 1, b = { 2, 3 } })
-- out == '{"a":1,"b":[2,3]}'

-- Encode pretty
local pretty = J.encode({ a = 1 }, { indent = 4 })

-- Empty array vs empty object
J.encode({})           --> "{}"
J.encode(J.empty_array) --> "[]"
J.encode({ items = J.empty_array }) --> '{"items":[]}'
```

null, empty_array, as_array

Lua cannot store nil in a table (the key disappears). `babet.json.null` fills the gap :

- **Decoding** : a JSON null becomes the `babet.json.null` sentinel (test with `==`), not `nil` — so the key stays present and visible to `pairs()`.
- **Encoding** : `babet.json.null` encodes as `null`.

`babet.json.as_array(t)` tags `t` (via an internal metatable) so that `encode` serialises it as a JSON array, even when the structure is ambiguous. Sentinels cannot be tagged (`as_array(json.null)` raises). Tables produced by decoding an empty JSON array already carry the tag : the round-trip `decode("[]") → encode` gives back `[]`.

Maximum nesting depth : **1000 levels** both ways (beyond : explicit (`nil, err`)).

Error contract

- **encode** : (`nil, err`) on encoding errors (cycle in the graph, value that doesn't map to JSON like a function or userdata, NaN/Inf numbers, etc.).
- **decode** : (`nil, err`) on parse error. The error message includes the line/column where parsing failed.
- **Wrong argument types** → raises via `luaL_error`.

Design decisions

- **empty_array sentinel** instead of guessing from the table structure (some libraries treat `{1, 2, 3}` as array and `{a=1}` as object, which is correct, but `{}` is ambiguous). An explicit sentinel removes all surprise.
- **NaN/Inf rejected**, not silently encoded as `null`. Different consumers handle JSON `null` differently for numeric fields ; better to fail loudly and let the caller decide.
- **No streaming API**. Round-trips are bounded by your script's memory regardless ; if you need to handle JSON larger than RAM, reach for a real streaming parser in a separate process.

Not in v1

- Streaming encoder/decoder.
- A schema validator (use a pure-Lua one — they're trivial to add at the script level).

logging — leveled logger (Lua module)

A standard leveled logger : `debug`, `info`, `warn`, `error`, `fatal`. Bundled as a pure-Lua module, loaded via `require("logging")`. Not in the `babet` namespace — it's a library module, like `inspect`.

Why

Every non-trivial script ends up reinventing some flavour of “levels + timestamps + optional file output”. Standardising it removes the bikeshed and makes log output consistent across Babet scripts.

API

```
local log = require("logging")
```

Function	Returns
<code>log.trace(...)</code> , <code>log.debug(...)</code> , <code>log.info(...)</code> , <code>log.warn(...)</code> , <code>log.error(...)</code> <code>log.set_level(lvl)</code>	nothing — each emits if its level is the current threshold. Never raise (even if the sink breaks) <code>lvl</code> = name (<code>"trace"..."error"</code> , case-insensitive) or constant (<code>log.INFO</code>). Unknown → raises
<code>log.set_output(out)</code>	<code>out</code> = any object answering :write (<code>io.open</code> handle, custom table). Default : <code>io.stderr</code> . Wrong type → raises
<code>log.set_color(bool)</code>	enables/disables ANSI colors (default : off). Strict non-boolean → raises

Function	Returns
<code>log.TRACE ... log.ERROR</code>	numeric level constants (10, 20, 30, 40, 50)

An earlier version of this page documented a **fatal** level that never existed (calling `log.fatal` crashes : nil call) and omitted `trace` and `set_color`. The five real levels : `trace=10`, `debug=20`, `info=30` (default threshold), `warn=40`, `error=50`.

Messages are prefixed `YYYY-MM-DD HH:MM:SS [LEVEL]` (level padded to 5 characters : `[INFO ]`, `[ERROR]`). Multiple arguments are joined with spaces, like `print`. Colors (opt-in via `set_color(true)`) : trace grey, debug cyan, **info uncolored** (the nominal case stays neutral), warn yellow, error red.

Quick example

```
local log = require("logging")
log.set_level("info")  -- trace and debug become no-ops

log.info("starting, pid =", babet.pid())
log.warn("attempt", 3, "of", 5)
log.error("connection failed:", err)

-- Optional : route to a file (a :write-able object)
local f = io.open("/var/log/myapp.log", "a")
if not f then
    log.error("cannot open log file; staying on stderr")
else
    log.set_output(f)
end
```

Output :

```
2026-06-11 14:32:01 [INFO ] starting, pid = 12345
2026-06-11 14:32:05 [WARN ] attempt 3 of 5
2026-06-11 14:32:05 [ERROR] connection failed: timeout
```

Error contract

- **set_level / set_output / set_color** : invalid argument (unknown level, object without `:write`, non-boolean) → **raises** via `error()` — that's a programmer bug, not a runtime condition.
- **The emit functions never raise** : the write is wrapped in a `pcall` — a sink that breaks midway (disk full, closed file) loses the message silently, never crashes the script.

Design decisions

- **Pure-Lua module**. No C++ needed, and being in Lua means scripts can monkey-patch it (e.g. add JSON-format output) at the call site without rebuilding Babet.
- **Five levels, trace to error — no fatal**. `error` plus an explicit `os.exit(1)` beats a level that kills the process quietly ; and five levels are enough for almost everyone.
- **Colors are opt-in, info always neutral**. The nominal case shouldn't shout ; only deviations (warn, error) stand out.
- **Single global sink**. Multiple loggers / hierarchical loggers / per-module levels are a feature creep trap. If you need multiple destinations, write a wrapper.

Not in v1

- JSON / structured logging output. Easy to bolt on at the script level if needed.
- Log rotation. Use `logrotate(8)` on the file output.
- Async / buffered output. Premature optimisation for typical use.

babet.signal — POSIX signals

Register Lua callbacks for POSIX signals (SIGTERM, SIGINT, SIGHUP, ...) so long-running scripts can shut down gracefully or reload configuration on demand.

Why

Long-running Babet scripts (bots, daemons, watchers) need to handle signals properly :

- SIGTERM from `systemctl stop` should trigger a clean shutdown.
- SIGINT from Ctrl-C should do the same.
- SIGHUP is the conventional “reload config” signal.

Without explicit handlers, these signals just kill the process, leaving open files, half-written databases, and orphaned children.

API

Function	Returns
<code>babet.signal.handle(name, fn)</code>	<code>(true, nil) (nil, err)</code> — installs the Lua callback
<code>babet.signal.ignore(name)</code>	<code>(true, nil) (nil, err)</code> — sets SIG_IGN
<code>babet.signal.default(name)</code>	<code>(true, nil) (nil, err)</code> — back to the OS default

An earlier version of this page also documented `kill`, `list` and `is_pending` : they don't exist — see “Not in v1”.

Signal names accepted (string, case-sensitive) :

"TERM", "INT", "HUP", "USR1", "USR2", "PIPE" — the six v1 signals (an earlier version of this page listed fourteen ; extending the table is trivial, see “Not in v1”). KILL and STOP can never be there — POSIX forbids catching them.

Quick example

```
local sig = babet.signal
local running = true

sig.handle("TERM", function()
    print("got SIGTERM, shutting down")
    running = false
end)
sig.handle("INT", function()
    print("got SIGINT, shutting down")
    running = false
end)
sig.handle("HUP", function()
    print("got SIGHUP, reloading config")
    cfg = load_config()
end)

while running do
    local line, err = sock:recv_line(1.0)
    if not line then
        if err == "interrupted" then
            -- A handled signal arrived during recv ; the callback
            -- already fired. Re-check the loop condition.
        else
            break
        end
    end
end
```

```

else
    process(line)
end
end

print("clean exit")

```

How callbacks run

When a signal arrives :

1. The kernel sets a pending flag (no Lua code runs from the signal handler itself — async-signal-safe restrictions).
2. If the script is in a blocking call (`recv`, `sleep`, `inotify.read`, `accept`, ...), the call returns `(nil, "interrupted")`.
3. Before returning, Babet dispatches all pending callbacks in registration order, in the main Lua thread.
4. The script continues normally — the callback can have modified globals (`running = false` is the typical pattern).

This means callbacks always run in Lua context, never from inside a signal handler. They can use any Lua features (no async-signal- safety restrictions).

Interaction with `debug.sethook` (v21 audit) : outside blocking calls, dispatch relies on a Lua debug hook (`lua_sethook`, fired periodically by instruction count). Consequences : a `debug.sethook(...)` installed by the script **overwrites** this mechanism — callbacks will then only be dispatched when the script enters a babet blocking call — and conversely, the first `signal.handle(...)` replaces any user hook already installed. Avoid mixing `babet.signal` and `debug.sethook` in the same script.

Error contract

- **Unknown signal name** → `(nil, "signal: unknown name 'XYZ'")`.
- **Calls from a worker thread** → **raise** via `luaL_error` ("signal.handle: signal handlers can only be configured from the main thread, not from a worker"). It's a structural bug in the script, not a runtime condition — hence the raise instead of `(nil, err)`. Signals are process-global ; only the main thread manages them.
- **Wrong argument types** → raises via `luaL_error`.
- **The callback receives no arguments** (install one callback per signal if you need to tell them apart) and **its errors are silently swallowed** : wrap your code in `pcall` if you need to observe them.

Design decisions

- **Callbacks run in the Lua main thread, not from the signal handler.** POSIX async-signal-safety rules forbid most useful operations from inside a real handler. By marking the signal as pending and dispatching from the next safe point, callbacks can do anything Lua can do.
- **Blocking calls return "interrupted" on a handled signal.** Without this, a `recv_line` waiting on the network would block until timeout regardless of `SIGTERM` arriving. With it, the shutdown can complete in milliseconds.
- **No re-entrance.** While a callback is running, additional signals are queued and dispatched after.
- **Forbidden in worker threads.** Signals are process-wide ; only one thread can sensibly own them. Workers must use channels for inter-thread communication.

Not in v1

- `kill(pid, name)`, `list()` and `is_pending()` — an earlier version of this page wrongly presented them as available. Workarounds : sending a signal = `babet.exec("kill", { "-TERM", tostring(pid) })` ; the list of supported names is above.
- Extending the signal table (`QUIT`, `ALRM`, `CHLD`, `WINCH`, ...) — trivial if a real need shows up.
- `sigprocmask` / fine-grained signal masking. Not commonly needed at script level.
- `signalfd`-based polling integration. The current dispatch model is simpler and covers the typical cases.

babet.socket — TCP client and server

Low-level TCP sockets with timeouts, signal awareness, and a small set of higher-level helpers (`recv_line`, `recv_all`). Foundation for any custom protocol — the IRC bot, a JSON-over-TCP daemon, etc. See [tls](#) for the encrypted variant.

Why

For everyday HTTP needs there's [http](#). But many scripts need to speak a custom line-oriented protocol (IRC, Redis, plain telnet-style daemons), where a real TCP socket with timeouts is the right tool. Lua has no built-in TCP, and `LuaSocket` is a separate dep ; `babet.socket` makes the common patterns one-liners.

API

Constructors

Function	Returns
<code>babet.socket.connect(host, port, timeout?)</code>	<code>socket</code> <code>(nil, err)</code>
<code>babet.socket.listen(host, port, backlog?)</code>	<code>server_socket</code> <code>(nil, err)</code>

- `timeout` : seconds (float), optional — default infinite (blocking). This is the budget for the TCP attempts **after DNS resolution**: a **global** deadline across all A/AAAA addresses. If DNS returns several addresses (typical dual-stack), successive attempts share the same budget — never `timeout` per address. On expiry: `(nil, "timeout")`. Known limit: DNS resolution itself (`getaddrinfo`, synchronous) is not covered by this deadline nor interruptible by signal — a slow resolver can add its own delay upfront.
- `backlog` : integer > 0, optional — default 16. `listen(2)` queue size.

TLS : `babet.socket.connect_tls(host, port, opts?)` and `s:starttls(opts?)` also exist — they have their own page, see [tls](#).

A `connect` — with or without a timeout — interrupted by a signal handled via [signal](#) returns `(nil, "interrupted")` after dispatching the callback, like `recv/accept`.

Methods (client socket)

Method	Returns
<code>s:send(data)</code>	<code>integer</code> bytes sent <code>(nil, err)</code> — sends everything (internal loop over partial writes)
<code>s:recv(n, timeout?)</code>	<code>string</code> <code>(nil, "timeout")</code> <code>(nil, "interrupted")</code> <code>(nil, err)</code>
<code>s:recv_line(timeout?)</code>	<code>string</code> (without <code>\n</code>) <code>(nil, ...)</code>
<code>s:recv_all(timeout?)</code>	<code>string</code> (read until EOF) <code>(nil, ...)</code>
<code>s:set_timeout(seconds)</code>	sets default timeout (<code>0</code> = infinite). NaN/Inf/negative → <code>(nil, err)</code> ; internal 1 ms floor for tiny positive values
<code>s:close()</code>	idempotent
<code>s:peer()</code>	<code>table {host, port}</code> <code>(nil, err)</code> — remote endpoint
<code>s:sockname()</code>	<code>table {host, port}</code> <code>(nil, err)</code> — local endpoint

`recv_line` reads bytes until `\n` (LF) is found. CR is stripped if present (`\r\n` → returned without the trailing `\r`). Has a hard 8 MiB cap to prevent DoS via an endless single line.

Methods (server socket)

Method	Returns
<code>srv:accept(timeout?)</code>	socket (nil, ...)
<code>srv:close()</code>	idempotent

Quick examples

TCP echo client

```
-- 5 s timeout for the WHOLE connect (all DNS addresses combined);
-- omit it for a blocking connect.
local s = assert(babet.socket.connect("localhost", 4000, 5))

s:send("hello\n")
local reply, err = s:recv_line()
print("got:", reply)
s:close()
```

Simple line-protocol server with graceful shutdown

```
local sig = babet.signal
local srv = assert(babet.socket.listen("0.0.0.0", 4000))
local running = true
sig.handle("TERM", function() running = false end)
sig.handle("INT", function() running = false end)

while running do
    local client, err = srv:accept(1.0)      -- 1 s timeout
    if not client then
        if err == "timeout" or err == "interrupted" then
            -- loop and re-check `running`
        else
            io.stderr:write("accept: ", err, "\n"); break
        end
    else
        repeat
            local line = client:recv_line(30)
            if line then client:send("you said: " .. line .. "\n") end
        until not line
        client:close()
    end
end
srv:close()
```

Error contract

- **Connect errors** (DNS, refused, network) → (nil, "socket: ..."). Timeout expiry → bare (nil, "timeout"). Handled signal during connect → (nil, "interrupted").
- **Bind errors** (port in use, permission) → (nil, "socket: ...").
- **recv** :
 - string (any length up to n requested).
 - Empty string "" when the peer closed cleanly. Not an error.
 - (nil, "timeout") if no data within timeout.
 - (nil, "interrupted") if a handled signal arrived.
 - (nil, "line too long") for `recv_line` past the 8 MiB cap.
 - (nil, err) on other errors.

- **send** : may return fewer bytes than requested. Wrap in a loop if you need to send a fixed amount (or use `recv_all`-symmetric semantics in your protocol).
- **Methods after close** → `(nil, "socket: closed")`.
- **NaN/Inf timeouts**, negative timeouts → raises via `luaL_error`.

Design decisions

- **Single-threaded, blocking I/O**. No `select/poll` exposed — use `workers` for concurrent connections, or short timeouts in a polling loop for simple cases.
- **recv_line has an 8 MiB cap**. Protects against a peer that sends megabytes without ever sending `\n`. Configurable later if a use case needs more.
- **Signal-aware blocking**. All `recv*`, `accept` — and `connect`, including without a timeout — return `"interrupted"` on a handled signal, so scripts can exit cleanly.
- **connect timeout is global, not per-address** (v21 audit). The deadline is created once for the whole call; a dual-stack host whose first address stalls doesn't multiply the delay. Accepted trade-off: a fallback address only gets the remaining budget.
- **recv returns "" on clean close, not nil**. Lets scripts distinguish “remote went away” (“”) from “no data yet” (“timeout”) without inventing yet another sentinel.

Not in v1

- UDP sockets. Easy to add additively (`babot.socket.udp.*`).
- Unix-domain sockets. Same reasoning.
- Native multiplexing (`select/poll/epoll` exposed to Lua). Use workers or short-timeout polling instead.

babot.sqlite — embedded SQL database

Wraps SQLite 3.53.1 (vendored), the most-deployed database engine in the world. Zero-config persistence for any script that needs more than a JSON file but less than a database server.

Why

A script that needs to keep state across runs (a bot's seen-list, a scraper's progress, accumulated metrics) shouldn't have to pull in PostgreSQL or invent its own file format. SQLite is exactly right at this scale : single file, transactional, fast, no daemon.

The Lua API mirrors the SQLite C API closely (`open` / `exec` / `prepare` / `step` / `finalize`) with safer defaults and Lua-friendly error returns.

API

Opening and closing

Function	Returns
<code>babot.sqlite.open(path, opts?)</code>	<code>db (userdata) (nil, err)</code>
<code>db:close()</code>	<code>(true, nil)</code> — idempotent

`opts` :

Field	Type	Default
<code>wal</code>	boolean — enable WAL journal mode	<code>false</code>
<code>busy_timeout</code>	integer ms — automatic retry for this long on a locked db	<code>0 = none</code> (immediate <code>SQLITE_BUSY</code>)

An earlier version of this page also documented `opts.readonly` and `opts.foreign_keys` : they don't exist — see “Not in v1”.

Special path `":memory:"` opens an in-memory database (lost on close). Use for tests or transient processing.

Execution and queries

Function	Returns
<code>db:exec(sql)</code>	<code>(true, nil) (nil, err)</code> — multi-statement accepted
<code>db:exec(sql, params)</code>	same, with bound parameters
<code>db:query(sql, params?)</code>	<code>stmt (iterator) (nil, err)</code> — one statement only

`db:query` returns a **callable iterator**, not a table : each call yields the next row (a table keyed by column names), then `nil` once exhausted — the idiomatic use is `for row in db:query(...) do`. Resources are released on exhaustion or garbage collection ; `stmt:close()` releases earlier (iteration abandoned midway).

Multi-statement SQL passed to `query` returns `(nil, "sqlite: query supports only one statement; ...")` — for multi-statement SQL, use `exec`.

Type mapping

SQLite type	Lua type
INTEGER	integer
REAL	number (float)
TEXT	string
BLOB	string (binary-safe)
NULL	read : key absent from the row (<code>row.col == nil</code> , but <code>pairs()</code> won't see it) ; write : use the SQL literal <code>NULL</code> (no sentinel in v1)

Quick examples

```
local db = assert(babet.sqlite.open("state.db", { wal = true }))

-- Schema (multi-statement : exec)
db:exec([[
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT UNIQUE NOT NULL,
        active INTEGER DEFAULT 1
    );
]])

-- Insert with parameters
assert(db:exec("INSERT INTO users (name) VALUES (?)", { "alice" }))

-- Last rowid : via SQL (no dedicated method in v1)
for row in db:query("SELECT last_insert_rowid() AS id") do
    print("inserted with id", row.id)
end

-- Query : query returns an ITERATOR
for row in db:query("SELECT id, name FROM users WHERE active = ?",
    { 1 }) do
```

```

    print(row.id, row.name)
end

-- Transaction : via SQL (no wrapper in v1)
db:exec("BEGIN")
local ok1 = db:exec("UPDATE users SET active = 0 WHERE name = ?",
                    { "alice" })
local ok2 = db:exec("UPDATE users SET active = 1 WHERE name = ?",
                    { "bob" })
if ok1 and ok2 then
    db:exec("COMMIT")
else
    db:exec("ROLLBACK")
end

db:close()

```

Error contract

- All runtime errors → (nil, "sqlite: <description>") with the SQLite error code in the message.
- **exec(sql) with placeholders but no params argument** → (nil, "sqlite: SQL contains placeholders but no params table provided; ...") — an explicit error instead of silently binding NULL. The guard applies to **every** statement of a multi-statement SQL string (v21 audit).
- **Supported placeholders** : ? (positional, bound from params[1], params[2], ...) and :name / @name / \$name (named, bound from params.name — the prefix is ignored). Numbered **?NNN placeholders are not supported** : the binder would treat them as a “named” parameter whose name is "NNN" (string key), a trap more than a feature. Use ? or :name.
- **Sparse keys in params** (e.g. {[1] = "a", [3] = "c"}) → (nil, err).
- **Empty BLOB string** → stored correctly as empty BLOB (previously buggy in pre-1.5).
- **Methods after close** → (nil, "sqlite: connection closed").
- **Wrong argument types** → raises via luaL_error.

Design decisions

- **WAL is opt-in, not default.** WAL is strictly better for most use cases, but it creates *-wal and *-shm sidecar files that some users find surprising. Opt-in keeps the default surprise-free, but every long-running daemon should pass wal=true.
- **Placeholders without params is an error**, not a silent NULL bind. Catches a class of injection-adjacent bugs early.
- **Errors are returned, not raised.** Even malformed SQL returns (nil, err). Scripts that don’t check are noisy but not catastrophic.

Not in v1

The following items were wrongly presented as available in an earlier version of this page — they belong to the original design and are **not implemented** : db:prepare / stmt:exec / stmt:finalize (reusable prepared statements — query re-prepares on each call), db:transaction(fn) / db:in_transaction() (workaround : exec("BEGIN"/"COMMIT"/"ROLLBACK"), see example), db:last_insert_rowid() / db:changes() (workaround : SELECT last_insert_rowid() / SELECT changes()), opts.readonly / opts.foreign_keys (workaround : PRAGMA foreign_keys = ON via exec), and the db.NULL write sentinel.

- BLOB streaming I/O (sqlite3_blob_open). Use string serialisation if you fit in memory.
- Virtual tables / FTS5 / R-Tree. Available via raw SQL if the vendored SQLite is built with them ; no Lua-level API yet.
- Backup API (sqlite3_backup_init). For now, VACUUM INTO 'backup.db' works as a one-liner.

For a higher-level ORM-like layer, build it in Lua on top of this API.

babet strings — string manipulation

A single function : split a string by a separator into a table. Predates the sub-namespace convention, lives directly on `babet`.

Why

Splitting a string by a delimiter is one of the most common string operations, and Lua’s standard library doesn’t have one out of the box (`string.gmatch` works but requires pattern escaping). A dedicated function is more discoverable and avoids the pattern-escaping trap.

API

Function	Returns
<code>babet.split(s [, sep [, max_splits]])</code>	<code>table</code> (array) of substrings

- `s` : the string to split. Binary-safe: NUL bytes are preserved.
- `sep` : optional separator — a **single character**, literal (no patterns). An empty string, or omitting the argument, switches to **character mode**: `s` is split into individual characters. More than one character → raises.
- `max_splits` : maximum number of cuts, optional (default -1 = unlimited). The uncut remainder lands in the last element, so 0 returns { `s` }. Ignored in character mode.

If `sep` doesn’t appear in `s`, the result is a one-element table containing the whole `s`.

Empty-string edge cases (historical behavior, frozen and tested):

- `babet.split("", sep)` returns { "" } — one empty entry, not an empty table.
- `babet.split("")` (character mode) returns {} — an empty table.

Quick example

```
local parts = babet.split("a,b,c,d", ",")
-- parts == { "a", "b", "c", "d" }

local one = babet.split("hello", ",")
-- one == { "hello" }

-- Character mode (sep omitted or empty)
local chars = babet.split("abc")
-- chars == { "a", "b", "c" }

-- Bounded number of cuts: remainder in the last element
local kv = babet.split("key,val,ue", ",", 1)
-- kv == { "key", "val,ue" }
```

Error contract

- **Wrong argument types** → raises via `luaL_error`.
- **sep longer than one character** → raises.
- **max_splits not an integer or < -1** → raises.
- Otherwise always succeeds.

Design decisions

- **Literal separator, not Lua pattern.** The common case is splitting CSV-like data, where you want `.` to mean a literal dot, not “any character”. If you need pattern matching, fall back to `string.gmatch`.

- **Empty entries are preserved.** "a,,b" splits into {"a", "", "b"}. Consumers who want to filter empty strings do it in one line of Lua.
- **split("", sep) returns { "" }.** This is the natural consequence of the “empty entries are preserved” rule (zero separators found → one element, the whole string — which happens to be empty). Observable since v1, so frozen rather than changed.
- **Character mode when sep is omitted or empty.** There is no default separator: without one, the only coherent reading of a split is character by character. Note: the split is per **byte**, not per Unicode code point — a multi-byte UTF-8 character will be torn apart.

Not in v1

- Pattern-based splitting (with escape rules). Use `string.gmatch` if needed.
- Multi-character separators.
- `joinTable(t, sep)` mirror — `table.concat` already exists in the stdlib.

babet sys — system utilities

Process and host introspection helpers : PID, hostname, kernel info, executable lookup, environment variables.

Why

Lua’s standard library has `os.getenv` and `os.execute`, but nothing for the PID, hostname, kernel info, or a portable `which`. These are universal scripting needs that don’t deserve a roundabout via `popen` calls.

These functions live directly on **babet** (flat, no sub-table) because they predate the per-module convention used by recent additions.

API

Function	Returns
<code>babet.pid()</code>	<code>integer</code> — current process ID
<code>babet.hostname()</code>	<code>string</code> <code>(nil, err)</code> — system hostname
<code>babet.uname()</code>	table with <code>sysname</code> , <code>nodename</code> , <code>release</code> , <code>version</code> , <code>machine</code>
<code>babet.which(cmd)</code>	<code>string</code> (full path from PATH) <code>(nil, "which: '<cmd>' not found in PATH")</code>
<code>babet.env(name)</code>	<code>string</code> <code>nil</code> — like <code>os.getenv</code> , but consistent
<code>babet.setenv(name, value)</code>	<code>(true, nil)</code> <code>(nil, err)</code>
<code>babet.getMemoryUsage()</code>	<code>integer</code> — Lua VM memory in bytes (after a full GC)
<code>babet.getDetailedMemoryUsage()</code>	<code>(integer, integer)</code> — currently both equal the GC count in bytes ; kept as two returns for API stability

Runtime constants

The same **babet** table also exposes constants describing the Babet binary that’s running the script. Useful for logging, gating features behind a minimum version, or sanity-checking the runtime.

Constant	Type	Example
<code>babet.VERSION</code>	<code>string</code>	"2.1.1"
<code>babet.VERSION_MAJOR</code>	<code>integer</code>	2
<code>babet.VERSION_MINOR</code>	<code>integer</code>	1
<code>babet.VERSION_PATCH</code>	<code>integer</code>	1

The string version is what `babet --version` prints. The integer components are there for programmatic comparison without parsing the string.

```

-- Log the runtime
print("running under Babet " .. babet.VERSION)

-- Gate a feature behind a minimum version
local need_minor = 1
if babet.VERSION_MAJOR < 2
  or (babet.VERSION_MAJOR == 2 and babet.VERSION_MINOR < need_minor) then
  error("this script needs Babet >= 2." .. need_minor)
end

```

Quick example

```

print("Running on:", babet.hostname(), "PID:", babet.pid())

local u = babet.uname()
print("Kernel:", u.sysname, u.release, u.machine)

-- Find a binary
local sh, err = babet.which("bash")
if sh then
  babet.exec({ sh, "-c", "echo hello" })
end

-- Read/write env
print("HOME:", babet.env("HOME"))
babet.setenv("MY_VAR", "value")

-- Memory introspection (forces a full GC first)
print("Lua VM memory:", babet.getMemoryUsage(), "bytes")
local a, b = babet.getDetailedMemoryUsage()
print("Detailed:", a, b)

```

Error contract

- **pid()** : never fails, always returns an integer.
- **hostname()** / **which()** : (value, nil) on success, (nil, err_string) on failure.
- **env(name)** : value if set, nil if unset. Only raises on an argument not convertible to a string (nil, table...) — a number is converted (env(42) looks up "42").
- **setenv(name, value)** : (true, nil) on success, (nil, err) on failure (rare — usually OOM or invalid name).
- **uname()** : never fails on any supported system, always returns the full table.
- **Wrong argument types** (e.g. which({})) → raises via `luaL_error` after string coercion as per Lua convention. Pass a table or boolean to force a real error.

Design decisions

- **env(name)** returns **nil**, not **""**, for **unset variables**. Matches `os.getenv` semantics. Tests should use `env("X") == nil`, not `env("X") == ""`.
- **which** returns the **absolute path**, not just “yes/no”. This is more useful : you can `exec` the returned path directly. For a boolean check, use `which(cmd) ~= nil`.
- **uname()** returns a **table**, not **multiple values**. Makes it forward-compatible if a field is ever added (e.g. `domainname`).
- **setenv and workers don’t mix** (v21 audit ; rule now **enforced by the runtime** since the cross-review). POSIX does not synchronize `setenv/getenv` across threads : a `babet.setenv` while a worker — or the runtime itself (DNS resolution, locale, `exec`) — reads the environment is a data race. `babet.setenv` (and `babet.chdir` — the working directory is also shared process-wide) therefore return (nil, err) once a `workers.spawn` has happened — permanently, even after `join`, even if the spawn later failed. The transition is serialized by the

same lock as the mutations : no worker can be born during a `setenv/chdir`. Set environment and working directory **before** the first worker.

Not in v1

Additive — could be added later :

- `unsetenv` — removing an inherited variable is not possible in v1 (`setenv(name, nil)` **raises** : the value must be a string ; an empty value is still a defined variable). And since the workers lock, any env mutation is forbidden after the first `spawn` anyway.
- `getuid` / `getgid` / `getppid` / `getlogin`.
- Full environment dump as a table.

For user lookups (UID → name, etc.) see the dedicated `user` module — that uses NSS, which is the right backend for that question.

babet tables — table manipulation helpers

Two helpers that fill obvious gaps in the Lua standard library : a proper deep copy, and a multi-source merge.

Why

Lua's standard library has no built-in deep copy and no multi-source table merge. Both are routine needs : merging config from defaults + overrides, snapshotting a table before mutation, copying a tree of options. Writing them per-script is repetitive and error-prone (missing the metatable, cycles, etc.).

These two functions live directly on the `babet` table (no sub-namespace) because they predate the per-module convention used by recent additions.

API

Function	Returns
<code>babet.mergeTables(t1, t2, ...)</code>	new table — string keys merged (last writer wins), numeric keys appended in order
<code>babet.deepCopyTable(t)</code>	new table , recursively copied

`mergeTables` is variadic — **two tables minimum** (fewer : raises), as many as you want beyond that. The inputs are not mutated. The merge has two rules :

- **String keys** : later tables overwrite earlier ones at the same key (last writer wins). Replacement is **shallow** : a table value overwrites the previous one **wholesale**, nested sections are not merged recursively. And values are **referenced, not copied** — the result shares its subtables with the inputs ; combine with `deepCopyTable` if you need an independent result.
- **Numeric keys (array part)** : concatenated in the order tables are passed, then in 1..n order within each table. So `mergeTables({1, 2}, {3, 4})` gives `{1, 2, 3, 4}`, not `{3, 4}`.

`deepCopyTable` copies nested tables recursively. Cycles are detected (no infinite loop), and shared subtables are copied only once (the structure is preserved, not duplicated). The **metatable** of each copied table is set as-is on the copy (shared reference — the metatable itself is not duplicated). Non-table values (numbers, strings, booleans, functions, userdata) are referenced as-is. Maximum depth : **75 levels** (beyond : raises "Table is too deep to copy (max depth 75 exceeded)").

Quick example

```
-- String keys : last writer wins
local defaults = { port = 8080, host = "localhost", debug = false }
local user_cfg = { port = 9000, debug = true }
```

```

local cfg = babet.mergeTables(defaults, user_cfg)
-- cfg.port == 9000      (user_cfg override)
-- cfg.host == "localhost"
-- cfg.debug == true

-- Numeric keys : append in order
local a = { 1, 2 }
local b = { 3, 4 }
local merged = babet.mergeTables(a, b)
-- merged == { 1, 2, 3, 4 }  (NOT { 3, 4 })

-- Deep copy : snapshot before mutation
local snapshot = babet.deepCopyTable(cfg)
cfg.port = 9999          -- does NOT affect snapshot
print(snapshot.port)     -- still 9000

```

Error contract

- **Wrong argument types** (passing a non-table to either function) → raises via `luaL_error`.
- Neither function returns errors via `(nil, err)`. They either succeed or raise.

Design decisions

- **Shallow values are referenced, not deep-copied**, in `deepCopyTable`. Copying functions or userdata makes no sense, and copying immutable values (numbers, strings, booleans) changes nothing. Only tables are recursively duplicated.
- **Metatables are not copied** in `deepCopyTable`. This is a conscious choice : preserving a metatable across a copy can surprise callers (the copy still triggers `__index` callbacks that mutate the original). If you need to copy with metatable, do it explicitly with `setmetatable(babet.deepCopyTable(t), getmetatable(t))`.
- **`mergeTables` is shallow**. If two source tables share a sub-table at the same string key, the result references the last one unchanged — it does not recurse into it. Combine with `deepCopyTable` if you need merge-then-isolate semantics.
- **Numeric keys are appended, not overwritten**. The choice comes from the most common Lua use case : combining list-shaped tables (`{1, 2} + {3, 4} → {1, 2, 3, 4}`). If you wanted overwriting semantics on numeric keys, build a destination table and assign explicitly.

Not in v1

Additive — could be added later without breaking SemVer :

- A `deepMergeTables` that recurses into nested tables when both sources have a table at the same key. Common request, but the semantics around lists vs maps gets ambiguous, hence not in v1.
- An `equalsTables` for deep structural comparison. Easy to write in pure Lua, hasn't surfaced as a real need yet.

babet time — monotonic and realtime clocks

Two clocks with sub-second precision and a sleep that honors `s/ms/us` units and signals. Fills the gaps in `os.time` / `os.clock`. The three historical functions live directly on `babet`; the date and duration utilities live under the `babet.time` sub-table (which also exposes the first three as aliases).

Why

Lua's `os.time()` only gives seconds, and `os.clock()` measures CPU time, not wall time. Neither is appropriate for measuring real durations (request latency, retry backoff, timeouts). And `os` has no portable sub-second sleep.

`babet.monotonic()` and `babet.now()` expose `CLOCK_MONOTONIC` (durations) and `CLOCK_REALTIME` (timestamps), with `babet.sleep()` honouring signal interruption.

API

Function	Returns
<code>babet.monotonic()</code>	<code>number</code> — seconds since arbitrary epoch (immune to clock changes)
<code>babet.now()</code>	<code>number</code> — POSIX time (seconds since 1970-01-01 UTC)
<code>babet.sleep(amount, unit?)</code>	<code>(true, nil) (nil, "interrupted")</code>

unit for `sleep` is one of "s" (default), "ms", "us". Floats are accepted. (No "ns": nanosleep doesn't guarantee that granularity anyway, and the code does not accept it.)

API — date and duration utilities (v1.8.0+)

Added under the `babet.time` sub-table. The three flat functions above are also aliased there (`babet.time.now`, `babet.time.monotonic`, `babet.time.sleep`) for ergonomy.

Function	Returns
<code>babet.time.iso(ts?)</code>	<code>string</code> — "YYYY-MM-DDTHH:MM:SSZ" (UTC). Without arg, formats the current time.
<code>babet.time.parse_iso(s)</code>	<code>(integer, nil) (nil, "parse_iso: ...")</code> — Unix timestamp in seconds.
<code>babet.time.parse_duration(s)</code>	<code>(integer, nil) (nil, "parse_duration: ...")</code> — number of seconds.
<code>babet.time.format_duration(n)</code>	<code>string</code> — "1d2h3m4s" style, compact, with zero components omitted.

ISO 8601 format accepted by `parse_iso` — pragmatic, but with one strict rule : **a timezone suffix is required**.

- Date and time separator : T or space.
- Timezone : Z (UTC), +HH:MM, or -HH:MM. **No** +0200, +02, or trailing absence — these are all rejected. Hours 00..23, minutes 00..59.
- Fractional seconds (.123) are accepted but ignored.
- A string without timezone is rejected explicitly rather than interpreted as UTC, because most real-world timestamps without a Z are local, not UTC, and silent misinterpretation is a common bug source.

Duration format accepted by `parse_duration` :

- Units : s, m (minute), h, d. No w, mo, y, ms, us, ns in v1.
- Compose unit chunks without whitespace : "1h30m", "2d12h", "45m30s", "1d1h1m1s".
- Units must appear in **strictly decreasing order** (d > h > m > s). "30m1h" is rejected.
- No duplicate unit : "1h1h" is rejected.
- No sign, no spaces. Empty string is rejected. A bare number without unit ("5") is rejected.
- "0s" is the only canonical representation of zero (matches `format_duration(0)`).

Round-trip invariant : for every integer `n >= 0`, `parse_duration(format_duration(n)) == n`. This is checked in the test suite over a representative sample.

Quick example

```
-- Measure a duration safely (immune to NTP / DST jumps)
local t0 = babet.monotonic()
do_something()
local elapsed = babet.monotonic() - t0
print(string.format("took %.3f s", elapsed))

-- Timestamp an event
```

```

local now = babet.now()
db:exec("INSERT INTO events (ts, kind) VALUES (?, ?)", { now, "ping" })

-- Sleep 100 ms, but wake up on a signal
local ok, err = babet.sleep(100, "ms")
if not ok and err == "interrupted" then
    print("woken by a signal")
end

-- ISO 8601 timestamp for logs (always UTC)
print("event at " .. babet.time.iso())          -- "2026-06-17T14:32:01Z"
print(babet.time.iso(0))                        -- "1970-01-01T00:00:00Z"

-- Parse a log line back to a Unix timestamp
local ts = assert(babet.time.parse_iso("2026-06-17T10:00:00+02:00"))
-- ts is 1781683200 (which is 2026-06-17T08:00:00Z)

-- Human-readable durations
local cache_ttl = babet.time.parse_duration("2h30m") -- 9000
print("cache valid for " .. babet.time.format_duration(cache_ttl))
-- "cache valid for 2h30m"

```

Error contract

- **monotonic()** / **now()** : never fail. Always return a number.
- **sleep(amount, unit)** :
 - (true, nil) if the sleep completed normally.
 - (nil, "interrupted") if a handled signal arrived during the sleep (see [signal](#)).
- NaN / Inf / negative amount → raises via `luaL_error`.
- Unknown unit string → raises via `luaL_error`.
- **iso(ts)** out of int64 bounds (|ts| beyond what the calendar can represent) → raises via `luaL_error` ("iso: value out of int64 range").

Design decisions

- **Two clocks, two functions, no flag.** `monotonic()` for durations, `now()` for timestamps. Conflating them in one function with a parameter would invite the wrong choice.
- **now()** for **realtime**, not `realtime()` — shorter, and the natural English meaning matches what’s expected (current time).
- **Signal-aware sleep.** A 60-second sleep that ignores `SIGTERM` is the classic graceful-shutdown bug. `sleep()` returns (nil, "interrupted") so the caller can exit cleanly.
- **No “format date” helper.** `os.date(format, babet.now())` works fine, and a duplicate function would just be a wrapper.

Not in v1

- `babet.boottime()` (`CLOCK_BOOTTIME` — includes suspend). Rarely needed, easy to add later.
- `babet.utcnw()` / `localnow()` helpers that return pre-formatted strings. Trivial in Lua, no value-add.

babet.socket TLS — **connect_tls** and **starttls**

The TLS half of [socket](#) : encrypted TCP for IRC over TLS, IMAPS, custom secure protocols. Two entry points : `connect_tls` for protocols that go TLS from the start (port 6697 IRC, 993 IMAPS, etc.) and `starttls` for protocols that upgrade an existing plain connection (SMTP, IMAP, IRC STARTTLS).

Why

Talking encrypted TCP without [http](#) is a real need : IRC bots over +6697, SMTP submission, custom internal services behind TLS. Doing it manually with `openssl s_client` via `exec` is unreliable ; `openssl` the C++ library + `cpp-httplib` chosen abstractions give a clean Lua API that handles cert verification correctly by default.

API

Function	Returns
<code>babet.socket.connect_tls(host, port, opts?)</code> <code>s:starttls(opts?)</code>	<code>tls_socket</code> <code>(nil, err)</code> <code>(true, nil)</code> <code>(nil, err)</code> — upgrade an existing plain socket

`opts` :

Field	Type	Default
<code>verify</code>	boolean — certificate verification	<code>true</code>
<code>ca_cert</code>	string (path to a CA bundle file)	system trust store
<code>ca_path</code>	string (path to a CA directory)	system trust store
<code>hostname</code>	string — SNI + name verification	<code>connect_tls</code> : the host argument. <code>starttls</code> : required if <code>verify = true</code>
<code>timeout</code>	number (seconds)	infinite — budget applied to each phase (TCP connect, then TLS handshake), not a single envelope ; DNS resolution is not covered

An earlier version of this page named the option `server_name` (the real name is `hostname`) and documented an `alpn` option that doesn't exist — see “Not in v1”.

A `tls_socket` has the same methods as a plain socket (`send`, `recv`, `recv_line`, `recv_all`, `set_timeout`, `close`, `peer`). The encryption is transparent.

Quick examples

Connect to IRC over TLS

```
local s = assert(babet.socket.connect_tls("irc.libera.chat", 6697, {
    timeout = 30,
}))
```

```
s:send("NICK babet-bot\r\n")
s:send("USER babet 0 * :babet bot\r\n")
```

```
while true do
    local line, err = s:recv_line()
    if not line then break end
    print(line)
    if line:match("^PING (.+)$") then
        s:send("PONG " .. line:match("^PING (.+)$") .. "\r\n")
    end
end
```

STARTTLS upgrade (SMTP submission)

```
local s = assert(babet.socket.connect("mail.example.com", 587))
print(s:recv_line())           -- 220 banner
s:send("EHLO myhost\r\n")
repeat
    line = s:recv_line()
until not line:match("^250%-")  -- last 250 line

s:send("STARTTLS\r\n")
print(s:recv_line())           -- 220 ready to start TLS

assert(s:starttls({ hostname = "mail.example.com" }))
-- s is now encrypted ; continue with EHLO + AUTH + ...
```

Self-signed server (dev / private CA)

```
local s = assert(babet.socket.connect_tls("internal.svc", 5555, {
    ca_cert = "/etc/myapp/internal-ca.crt",
}))
```

Skip verification (TESTING ONLY)

```
local s = assert(babet.socket.connect_tls("self-signed.local", 8443, {
    verify = false,
}))
```

Error contract

- All errors prefixed "tls: ..." (handshake, cert verify, etc.) or "socket: ..." (DNS, connect, timeout).
- (nil, err) always — no exceptions for “expected” failures like cert mismatch.
- **verify=true + invalid cert** → (nil, "tls: certificate verify failed: <reason>"). Common reasons : expired, self-signed, hostname mismatch, unknown CA.
- **starttls with verify = true and no opts.hostname** → (nil, "tls: starttls with verify=true requires opts.hostname; pass hostname or set verify=false") — a certificate cannot be verified without knowing which name to verify ; the explicit refusal beats a silently incomplete verification.
- **After a failed starttls** : the socket is handed back **as it was** — plaintext, blocking, still open (you can close cleanly or retry). Edge case : if restoring blocking mode fails (nearly impossible), the socket is **closed** and the error carries the suffix "(socket closed: could not restore blocking mode)" — a safe, explicit state instead of a socket with altered behaviour.
- **NaN/Inf timeouts**, wrong types → raises via `luaL_error`.

Trust store

Babet ships its own statically-linked OpenSSL. Out of the box, `verify=true` works on :

- **Arch, Debian, Ubuntu, Alpine, Gentoo** : via `--openssldir=/etc/ssl` baked into the vendored OpenSSL.
- **Fedora, RHEL, CentOS, Rocky, Alma, OpenSUSE, FreeBSD, NetBSD** via runtime probing of known CA bundle paths.

Overrides, in priority order :

1. `ca_cert` / `ca_path` per call.
2. `SSL_CERT_FILE` / `SSL_CERT_DIR` environment variables.
3. The two mechanisms above.

If none of these find a CA store and you use `verify=true`, the handshake fails with a clear error. See [security](#) for the full picture.

Design decisions

- **TLS 1.2 minimum.** SSL 3, TLS 1.0, TLS 1.1 are unconditionally rejected — they're broken, no modern server should rely on them. TLS 1.3 is negotiated automatically when available.
- **verify=true by default, verify=false requires explicit opt-in.** No way to accidentally turn off cert checking.
- **hostname defaults to the host argument** (for `connect_tls`), so SNI works out of the box. Pass it explicitly when connecting by IP to a TLS server that uses SNI — and it is required for `starttls` with verification, since the existing socket doesn't know which name to expect.
- **Same methods as plain socket**, so a function that takes a `socket` works for both transparently.

Not in v1

- ALPN negotiation (an earlier version of this page documented an `alpn` option that never existed).
- Client certificate authentication. Possible to add as a `client_cert` / `client_key` opt later.
- TLS session resumption / session tickets. Not commonly needed at script level.
- OCSP stapling verification. Reliance on OpenSSL's defaults.

babet.toml — TOML parser

Backed by [toml++](#), a strict TOML 1.0 parser. Decode only — encoding TOML faithfully from a Lua table is ambiguous (Lua doesn't distinguish between inline tables and dotted tables, for instance).

Why

TOML is the natural config-file format for modern tools (Cargo, pyproject, systemd-credentials, etc.). A Babet script reading its own config should be able to do it directly, not via a TOML → JSON conversion via `tomlq`.

API

Function	Returns
<code>babet.toml.decode(text)</code>	<code>table</code> (parsed TOML) <code>(nil, err)</code>

The returned table mirrors the TOML structure :

- Strings → Lua strings (UTF-8)
- Integers → Lua integers
- Floats → Lua numbers
- Booleans → Lua booleans
- Datetimes → Lua strings in ISO 8601 form (TOML has no Lua-native equivalent ; use `os.date` or a date library if you need parsed components)
- Arrays → Lua tables (1-indexed)
- Tables (`[section]` or `{ inline }`) → Lua tables (with string keys)

Quick example

```
local body = [[
title = "My App"
[server]
host = "localhost"
port = 8080
features = ["auth", "metrics"]

[database]
url = "sqlite:///var/lib/app.db"
]]
```

```

local cfg, err = babet.toml.decode(body)
if not cfg then error("config parse failed: " .. err) end

print(cfg.title)           -- "My App"
print(cfg.server.host)     -- "localhost"
print(cfg.server.features[1]) -- "auth"
print(cfg.database.url)    -- "sqlite:///var/lib/app.db"

```

Error contract

- **decode** : (nil, err) on parse error. The error message includes the line/column where parsing failed, and a short description of what was wrong (per toml++).
- **Non-string argument** → raises via `luaL_error`.

Design decisions

- **Decode only, no encode**. Round-tripping a TOML file via a Lua table loses formatting (comments, dotted vs inline tables, newlines between sections) that real users care about. If you need to write TOML, build the string yourself — TOML is designed to be human-writable. A faithful encoder would be a large undertaking for limited value.
- **Datetimes as strings**. TOML's datetime type is rich (offset, local, date-only, time-only) but Lua has no native equivalent. Returning a string in ISO 8601 form preserves the data losslessly and lets the script parse it with the tools it prefers.

Not in v1

- Encoder. May be added if a real use case appears, but unlikely.
- A schema validator. Build one in Lua if you need it ; the decoded structure is just a table.

babet.user

System user lookups via NSS, without parsing `/etc/passwd` directly.

Why

On a modern Linux system, users may come from multiple sources : `/etc/passwd`, LDAP, SSSD, NIS+, FreeIPA, systemd-userdb, or other NSS plugins. Reading `/etc/passwd` directly misses everything except the first source. `babet.user` is backed by `getpwnam_r(3)` and `getpwuid_r(3)`, which traverse the resolver chain configured in `/etc/nsswitch.conf` — exactly what `id` or `getent passwd` does.

API

Function	Returns
<code>user.get(name_or_uid)</code>	<code>table (nil, "user not found") (nil, "user: <err>")</code>
<code>user.exists(name_or_uid)</code>	<code>boolean</code>

`name_or_uid` accepts :

- a **string** → looked up by name via `getpwnam_r`
- a **non-negative integer** → looked up by UID via `getpwuid_r`

Any other type, a float, a negative integer, a string containing an embedded NUL byte, or an integer above `uid_t` max raise via `luaL_error` — these are caller bugs, not runtime conditions to handle gracefully.

Returned table on success

```
{
  name  = "yaourt",
  uid   = 968,
  gid   = 968,
  gecos = "yaourt AUR build user",
  home  = "/var/cache/yaourt",
  shell = "/usr/sbin/nologin",
}
```

All string fields are guaranteed non-nil. They may be empty strings when the underlying NSS source has no value (this is rare but possible — typically empty `gecos`).

Quick example

```
-- Ensure a service user is provisioned before launching the daemon.
```

```
if not babet.user.exists("yaourt") then
  error("yaourt user not provisioned — run useradd first")
end
```

```
local u = assert(babet.user.get("yaourt"))
babet.chdir(u.home)
```

Error contract

- **Bad argument type** (table, boolean, nil, float, negative integer, UID above `uid_t` max, string with embedded NUL) → raises via `luaL_error`. Use `pcall` if you need to recover.
- **User absent** (the resolver returned no match) → `get()` returns (nil, "user not found") ; `exists()` returns false.
- **NSS error** (resolver itself broken : LDAP unreachable, out of memory, etc.) → `get()` returns (nil, "user: <description>") ; `exists()` returns false (errors are silently coerced into “absent” for predicates ; use `get()` if you need to diagnose).

Design decisions

- **NSS only, never `/etc/passwd`** : reading the file directly would silently miss every account that isn't in the local file. The whole point of `babet.user` is that the script gets the same answer as `id` or `getent passwd`.
- **Thread-safe `_r` variants** : Babet exposes `babet.workers`, so we use `getpwnam_r/getpwuid_r` everywhere.
- **`gecos` exposed raw** : the historical format is `Full Name,Office,WorkPhone,HomePhone[,Other]`, but in practice most entries hold a free-form name. Parsing it in Lua is trivial if you need to ; baking a parser in C++ would be premature.
- **NUL byte rejection** : `"root\0evil"` would be seen as `"root"` by `getpwnam_r` (it stops at the first NUL). On user-derived input, that could bypass an upstream identity check. Explicit rejection is safer than implicit truncation.
- **`uid_t` range check** : on Linux, `uid_t` is `uint32_t`, while `lua_Integer` is `int64_t`. Without a check, `5_000_000_000` would be silently truncated to a 32-bit value and could match an unrelated account.

Not in v1

Additive — these could be added later without breaking SemVer :

- Group lookups (`getgrnam` / `getgrgid`) — will be added when a concrete use case asks for them.
- `/etc/shadow` access (passwords, password aging) — out of scope. Requires root and is rarely useful outside niche admin scripts. Password authentication should go through PAM, not Babet.
- User/group creation — already feasible via `babet.exec("useradd ...")` and `groupadd`. No need for a dedicated binding.

babot.workers — parallel jobs

Run Lua code on real OS threads. Useful for I/O-bound work that benefits from concurrency (fetching many URLs, scanning many files, processing batches) and CPU-bound work on multi-core hardware.

Why

Lua has coroutines, but those run cooperatively on a single OS thread — fine for state machines, not for using multiple cores. And many tasks are I/O-bound : 100 HTTP requests that each take 500 ms still take 50 s sequentially, but ~500 ms in parallel.

`babot.workers` spawns real OS threads each running their own isolated Lua state, communicating with the parent via passed-in arguments and return values. No shared state means no locks, no data races — just send work in, get results out.

API

Function	Returns
<code>babot.workers.spawn(code, args?, opts?)</code>	<code>job (userdata) (nil, err)</code>
<code>job:join()</code>	<code>(true, result) (false, err)</code> — blocking , see pitfall below
<code>job:poll()</code>	<code>string</code> — "running" "done" "error", non-blocking
<code>job:send(v, timeout?)</code>	<code>(true) (false, err)</code> — into the worker's inbox
<code>job:recv(timeout?)</code>	<code>(true, v) (false, err)</code> — from the worker's outbox
<code>job:close()</code>	closes both queues ; unblocks worker and parent

`opts : inbox_capacity / outbox_capacity` (integers, default 64) — message queue sizes.

`timeout` (seconds) for `send/recv` : omitted or `nil` = block indefinitely ; `0` = non-blocking (`err == "full" / "empty"`) ; otherwise bounded wait (`err == "timeout"`). A closed queue yields `err == "closed"`. On the worker side, `worker.send(v, timeout?)` and `worker.recv(timeout?)` are symmetrical.

join() pitfall : possible deadlock. `join()` waits for the worker to finish, with no deadline. If the worker is blocked in a `worker.recv()` without a timeout (waiting for a message that will never come), `join()` never returns — parent and worker wait on each other. Unlike the garbage collector (which closes the queues before joining), `join()` closes nothing. Practical rules : call `job:close()` before `join()` whenever the worker may be waiting for a message ; or use finite timeouts in `worker.recv()` ; and `job:poll()` to test without blocking.

code argument

A Lua source string that runs in the worker. The full `babot` namespace is available, plus a `worker` global with :

- `worker.args` — the second argument passed to `spawn`.
- `worker.send(v, timeout?) / worker.recv(timeout?)` — the message channel to/from the parent (same timeout conventions as the parent side, see table above).

The worker's last expression value (or return value) is what `join()` returns as `result`.

args argument

Any value that can be deep-copied between Lua states : `nil`, booleans, numbers, strings, tables of the same. **Not** : functions, userdata, threads, tables with cycles. Tables are deep-copied — no sharing. Maximum depth : **32 nesting levels** (beyond : `(nil, "workers: spawn: value too deeply nested")`). Same rules for the worker's return value and for `send/recv` messages.

Quick examples

Parallel HTTP fetches

```
local urls = {
    "https://a.example/",
    "https://b.example/",
    "https://c.example/",
    "https://d.example/",
}

local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, { timeout = 10 })
        if not r then return { ok = false, err = err } end
        return { ok = true, status = r.status, length = #r.body }
    ]], { url = url })
end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    if ok then
        print(i, result.status or result.err)
    else
        print(i, "worker crashed:", result)
    end
end
```

CPU-bound work with cancellation

Cancellation is **cooperative**, built on the message channel : the parent sends "stop", the worker checks its inbox periodically with `worker.recv(0)` (non-blocking).

```
local jobs = {}
for i = 1, 4 do
    jobs[i] = babet.workers.spawn([[
        local chunk = worker.args.chunk
        local total = 0
        for x = chunk.from, chunk.to do
            if x % 1000 == 0 then
                local got, msg = worker.recv(0) -- non-blocking
                if got and msg == "stop" then
                    return { cancelled = true, partial = total }
                end
            end
            total = total + heavy_compute(x)
        end
        return { total = total }
    ]], { chunk = { from = i * 1000, to = (i + 1) * 1000 - 1 } })
end

-- ... later, if needed :
-- for _, j in ipairs(jobs) do j:send("stop") end
-- then join() to collect the partial results.
```

Worker pool pattern

```
local function map_parallel(items, code, max_concurrent)
  -- no cpu_count() in v1 : pick a value, or detect :
  -- tonumber(babet.exec("nproc").stdout)
  max_concurrent = max_concurrent or 4
  local results = {}
  local i = 1
  local active = {}

  while i <= #items or next(active) do
    -- launch as many as we can
    while i <= #items and #active < max_concurrent do
      active[#active + 1] = {
        index = i,
        job = babet.workers.spawn(code, { item = items[i] }),
      }
      i = i + 1
    end
    -- harvest any finished
    for k = #active, 1, -1 do
      local entry = active[k]
      if entry.job:poll() ~= "running" then
        local ok, result = entry.job:join()
        results[entry.index] = ok and result or { err = result }
        table.remove(active, k)
      end
    end
    if next(active) then babet.sleep(10, "ms") end
  end
  return results
end
```

Error contract

- **spawn** : (nil, err) if the OS thread can't be created (rare — usually resource limits).
- **join** :
 - (true, result) — worker completed normally ; result is its return value (or nil if it didn't return).
 - (false, err) — worker crashed with an uncaught error ; err is the error message + traceback.
 - (false, "workers: join: result already consumed") — join already called on this job.
 - join() has **no** timeout : it blocks until the worker ends (see the deadlock pitfall above).
- **Wrong argument types** → raises via `luaL_error`.

Sharing data

Workers don't share Lua state. Communication is :

- **In** : via the `args` second argument to `spawn` (deep-copied into the worker's state).
- **Out** : via the worker's return value (deep-copied back).
- **Side effects** : a worker can write to a file, append to a log, query a database — but coordination through such side effects is the script's responsibility.

If two workers write to the same SQLite file, set `wal=true` on `open` so they don't lock each other out. SQLite handles the synchronisation correctly with WAL ; in `busy_timeout` you can configure how long to wait on a busy db.

Design decisions

- **One Lua state per worker, no sharing.** The alternative — shared state with locks — is a well-known source of subtle bugs and we don't think Lua-level locks are worth the design effort at this layer. Pure message-passing is simpler.
- **No global thread pool.** Each `spawn` creates a fresh OS thread, each `join` closes it. For tight loops, build a pool in Lua (see example above). The simpler model wins on legibility.
- **The first `spawn` locks `setenv` and `chdir`** — permanently, even after `join`. The environment and the working directory are process-wide state ; mutating them while a worker runs is a data race. Set them before the first worker. See the note in [sys](#).
- **`babet.signal` is not available in workers.** Signals are process-wide ; only the main thread can sensibly own them.

Not in v1

Note (v21 audit) : the items below belong to the original design and are **not implemented today** — the previous version of this page wrongly presented them as available : `job:join(timeout)`, `job:done()` (use `job:poll()`), `job:cancel()` / `worker.cancelled()` (cooperative cancellation can be built today with `job:send("stop")` + a periodic `worker.recv(0)`), and `babet.workers.cpu_count()`.

- Channels **between workers** (worker worker — the parent worker channel already exists : `send/recv`). Add later if a use case shows the pattern is common.
- Shared memory (`mmap`). Same.
- Async I/O futures (`spawn().then(...)` style). Out of scope ; use a polling loop with `job:poll()`.

Cookbook

Concrete recipes for common scripting tasks. Each recipe is short and self-contained ; copy, adapt, ship.

Watch a directory and email new files

A drop directory that mails out every file as it arrives. Uses `babet.inotify` for instant wakeup (no polling), and listens on `close_write` + `moved_to` so the file is guaranteed complete.

```
local w = assert(babet.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))

babet.signal.handle("TERM", function()
    w:close()
    os.exit(0)
end)

while true do
    local events, err = w:read()
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify: ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            -- queue overflowed: rescan the directory
            -- (events were lost)
        else
            local path = "/srv/incoming/" .. ev.name
            send_email(path)
            os.remove(path)
        end
    end
end
```

```

    end
end

```

Ensure a service user exists, then drop to it

A typical install-time check before launching a daemon as a non-root user.

```

local function ensure_user(name)
    if babet.user.exists(name) then return true end
    local ok, err = babet.exec("useradd", {
        "--system",
        "--home-dir", "/var/lib/" .. name,
        "--create-home",
        "--shell", "/usr/sbin/nologin",
        name,
    })
    if not ok then
        return nil, "useradd failed: " .. tostring(err)
    end
    return true
end

assert(ensure_user("myapp"))
local u = assert(babet.user.get("myapp"))
babet.chdir(u.home)

```

Fetch many URLs in parallel

babet.workers runs Lua on multiple cores. Total wall-clock is close to the slowest single fetch, not the sum.

```

local urls = { "https://a.example/", "https://b.example/",
    "https://c.example/", "https://d.example/" }

local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, { timeout = 10 })
        if not r then return { error = err } end
        return { status = r.status, length = #r.body }
    ]], { url = url })
end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    print(i, ok, result and result.status or result.error)
end

```

Graceful shutdown of a long-running script

Catch SIGTERM / SIGINT so the script can close resources properly. Works with `systemctl stop` and Ctrl-C.

```

local log = require("logging")
log.set_level("info")

local running = true
babet.signal.handle("TERM", function()
    log.info("SIGTERM, shutting down")

```

```

    running = false
end)
babet.signal.handle("INT", function()
    log.info("Ctrl-C, shutting down")
    running = false
end)

while running do
    -- main loop ; blocking calls return (nil, "interrupted") when
    -- a handled signal arrives, so the loop exits quickly.
    local line, err = sock:recv_line()
    if not line then
        if err == "interrupted" then break end
        log.error("recv: ", err); break
    end
    process(line)
end

sock:close()
log.info("clean shutdown")

```

Read a TOML config, validate, persist to SQLite

```

-- Load TOML
local f = assert(io.open("config.toml", "r"))
local body = f:read("a"); f:close()
local cfg, perr = babet.toml.decode(body)
if not cfg then error("config.toml: " .. perr) end

-- Validate (cheap manual checks ; for a real schema, build one in Lua)
assert(type(cfg.server) == "table", "missing [server]")
assert(type(cfg.server.host) == "string", "missing server.host")
assert(math.type(cfg.server.port) == "integer", "server.port must be integer")

-- Open db, persist
local db = assert(babet.sqlite.open("state.db", { wal = true }))
db:exec([[CREATE TABLE IF NOT EXISTS config (k TEXT PRIMARY KEY, v TEXT)])]
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
    { "host", cfg.server.host })
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
    { "port", tostring(cfg.server.port) })
db:close()

```

Cache with a human-readable TTL, expiry as ISO timestamps

Parse a TTL from config ("15m", "2h30m", "1d"), compute a deadline, and log it in a uniform UTC format that grep'ing across log files actually works on.

```

-- Config from TOML, CLI, env, anywhere.
local raw_ttl = "15m"

local ttl, err = babet.time.parse_duration(raw_ttl)
if not ttl then error("bad TTL '" .. raw_ttl .. "': " .. err) end

local deadline = babet.now() + ttl
print(string.format("[%s] cache entry created, expires at %s (TTL %s)",
    babet.time.iso(),

```

```

    babet.time.iso(deadline),
    babet.time.format_duration(ttl))

-- ...later, somewhere else in the script:
if babet.now() >= deadline then
    print "[" .. babet.time.iso() .. "] cache miss: entry expired"
end

```

Useful properties of this pattern :

- `babet.time.iso()` is always UTC, always 20 characters, always the same shape. `grep` and `sort` work on the timestamps directly.
- `parse_duration("15m")` returns an integer, so `now() + ttl` is exact (no float drift over long durations).
- `format_duration(ttl)` round-trips : `parse_duration( format_duration(n)) == n` for every `n >= 0`. Safe to use as a canonical representation in logs or databases.

Pretty-print a Lua value

The bundled `inspect` module gives readable output for nested tables.

```

local inspect = require("inspect")
print(inspect({ a = 1, b = { c = 2, d = { 3, 4, 5 } } }))
-- { a = 1, b = { c = 2, d = { 3, 4, 5 } } }

```

Hello there

The minimal Babet script :

```

babet.helloThere()

```